FeatureScalpel: Mechanistic Interpretability And Intervention Of Large Language Models via Sparse Autoencoder

A Main Project Report

Submitted to the APJ Abdul Kalam Technological University
in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering(AI)

by

ADITHYAN R S *(B23CS2109)*
ALONA BIJU *(B23CS2113)*
ALVIN BINU *(B23CS2114)*
DARSAN M *(B23CS2125)*

Department of Computer Science and Engineering
Mar Baselios College of Engineering and Technology
(Autonomous)
Mar Ivanios Vidya Nagar, Nalanchira
Thiruvananthapuram–695015

October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MAR BASELIOS COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
MAR IVANIOS VIDYA NAGAR, NALANCHIRA
THIRUVANANTHAPURAM–695015

CERTIFICATE

This is to certify that the report entitled **FeatureScalpel: Mechanistic Interpretability And Intervention Of Large Language Models via Sparse Autoencoder** submitted by **Adithyan R S** (KTU Reg No: B23CS2109), **Alona Biju** (KTU Reg No: B23CS2113), **Alvin Binu** (KTU Reg No: B23CS2114) and **Darsan M** (KTU Reg No: B23CS2125) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering (AI) is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Poorna B R
(Project Guide)
[TODO: designation]
Department of CSE

Dr. Ann Alex
(Head of the Department)
Professor
Department of CSE

Place: Thiruvananthapuram Date:

Declaration

We hereby declare that the project report entitled **FeatureScalpel: Mechanistic Interpretability And Intervention Of Large Language Models via Sparse Autoencoder**, submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI), is a record of original work carried out by us at the Department of Computer Science and Engineering, Mar Baselios College of Engineering and Technology (Autonomous), Thiruvananthapuram. The experimental results reported here were produced by the system described in this report. Wherever the work of others has been used, it has been duly acknowledged and cited. This report has not been submitted, in whole or in part, to any other University or Institute for the award of any degree or diploma.

Place: Thiruvananthapuram
Date:

Adithyan R S (B23CS2109)
Alona Biju (B23CS2113)
Alvin Binu (B23CS2114)
Darsan M (B23CS2125)

Acknowledgement

We express our deepest gratitude to our **Project Guide** from the Department of Computer Science and Engineering for continuous guidance, critical feedback and technical insight throughout this project. Their advice on experimental rigour and on stating results precisely shaped both the direction and the quality of this work.

We sincerely thank **Dr. Ann Alex**, Head of the Department of Computer Science and Engineering, for academic support, provision of resources and encouragement at every stage of the project.

We acknowledge the open-source communities behind **PyTorch**, **TransformerLens**, **SAELens**, **Hugging Face** and **Streamlit**, the author of the pretrained **gpt2-small-res-jb** sparse autoencoders, and the **Neuronpedia** project. We also thank the authors of ROME, whose openly released fact datasets, including CounterFact, formed the basis of the benchmarks, and the authors of the model-stitching work on which our translators build.

Finally, we are grateful to our families and friends for their patience and encouragement throughout this endeavour.

Adithyan R S
Alona Biju
Alvin Binu
Darsan M

Abstract

Language models often rank a correct answer below competing words even though the answer is within reach. Fixing this by changing weights is permanent and gets worse as edits pile up, and adding text to the prompt does not use what the model already computes. This project, **FeatureScalpel**, asks whether the fix can be **transient and weight-free**. During one forward pass the residual stream of GPT-2 Small is edited through the features of a pretrained **sparse autoencoder (SAE)**, and the edit disappears when the pass ends.

The report follows the work as it happened, mistakes included. Muting the target's own helpers corrected 1 of 22 test facts and muting the leading competitor corrected 3. Muting alone failed on our probe prompt, while muting together with boosting brought it to rank 1. Safety filtering, a study of depth, a batched implementation (186 model calls per prompt and layer down to 6), pool refill and candidates from all prompt positions followed. A small network that chose the strengths did no better per prompt than one learned pair of numbers, but tuning one multiplier per feature by gradient descent worked far better. On 300 held-out CounterFact prompts it brought the true answer to rank 1 on 271, against 161 for a sweep given the same range and 94 for the original settings, with lower side effects and 5 to 6.5 times less compute. An additive edit then pushed random, unrelated words to rank 1 on 20 of 20 prompts, so reaching rank 1 says little about what a model knows. To check our measures against an outside reference, we reproduced ROME's official implementation on GPT-2 Small, traced where its facts are decided, and scored our edit with ROME's own metrics. On those scores a weight edit generalises further to reworded prompts (98.5% against 67%, on different records), which agrees with our finding that the edit is a push. A side study applied the same tools to a new question: can an edit be carried from GPT-2 Small to GPT-2 Medium? A linear translator keeps the receiving model working. An exported edit lifts the target's median rank from 142 to 14 but reaches rank 1 on only 8% of records. What crosses is a push toward a word, not a fact. Mixing half of Medium's state into Small raises Small's accuracy from 8.5% to 9.5%. The side study's pass bars were fixed before the runs, and its limits (one model family, one dataset, and no head-to-head comparison with ROME or in-context editing on one record list) are stated.

Table of Contents

Acknowledgement i

Abstract ii

List of Figures vi

List of Tables vii

1 Introduction 1

1.1 Background and Motivation 1
1.2 How the Project Grew 1
1.3 Problem Statement and Research Questions 2
1.4 Objectives, Scope and Contributions 3
1.5 Organisation of the Report 4

2 Literature Review 5

2.1 Knowledge Editing 5
2.2 Locating Facts and Measuring Edits 5
2.3 Activation Steering and Sparse Autoencoders 6
2.4 Causal Localisation and Attribution 7
2.5 Gradient-Based Optimisation of Edits and Steering Vectors 7
2.6 Comparing and Stitching Representations 8
2.7 Research Gap 8

3 The Edit and How It Works 9

3.1 Setting and Notation 9
3.2 SAE Features and Delta Patching 9
3.3 Choosing Candidates: Causal Ranking 10
3.4 The Hybrid Mute and Boost Sweep 10
3.5 The Gradient-Descent Editor 10
3.6 The Additive Edit 12
3.7 Benchmark and Metrics 13

4 Evolution of the System: a Record of Trial and Error **14**
4.1 The Starting Point . . . 14
4.2 Single-Feature Ablation . . . 14
4.3 Several Competitors, Spread-Out Support . . . 15
4.4 Mute and Boost Together . . . 15
4.5 Where to Edit, and Making It Fast . . . 17
4.6 Making the Sweep Reliable . . . 17
4.7 Is the Layer, or the SAE, the Limit? . . . 20
4.8 Scaling Up: the CounterFact Benchmark . . . 20
4.9 Learning the Strengths . . . 21
4.10 Gradient Descent Enters . . . 22
4.11 Forcing Every Target to Rank 1: the Additive Edit . . . 24
4.12 What the Edit Does Beyond the Next Word . . . 25
4.13 Where the Main Line Stands . . . 27

5 ROME as an External Reference **28**
5.1 Why ROME Is in the Project . . . 28
5.2 What ROME Does . . . 29
5.3 How ROME Was Set Up Here . . . 30
5.4 Causal Tracing on GPT-2 Small . . . 32
5.5 Choosing ROME's Edit Layer . . . 34
5.6 How ROME's Scores Relate to Ours . . . 34
5.7 A Cross-Check: Our Edit Scored with ROME's Metrics . . . 35
5.8 What Remains and What the Paper Needs . . . 36

6 Applied Study: Carrying an Edit to Another Model **38**
6.1 The Question and the Design . . . 38
6.2 The Channel Works . . . 40
6.3 Export: a Real but Weak Effect . . . 40
6.4 Searching for a Better Translator . . . 41
6.5 What Is Carried: a Fact or a Push? . . . 43
6.6 Import: Medium's State into Small . . . 43
6.7 What the Side Study Shows . . . 45

7 Implementation **46**
7.1 Software, Hardware and Application . . . 46
7.2 Correctness Checks and Engineering Lessons . . . 47
7.3 Reproducibility and Development Process . . . 48

8 Discussion, Conclusion and Future Work **50**
8.1 What the Work Shows 50
8.2 Threats to Validity and What Is Not Shown 51
8.3 Conclusion 51
8.4 Decisions for the Research Paper 52
8.5 Future Work 53

References **54**

Appendices **59**
Appendix A: Research Journal and Report Sections 59
Appendix B: Pass Bars of the Side Study and Their Verdicts 59
Appendix C: Reproducing the Results 60
Appendix D: Where Every Figure Comes From 61

List of Figures

1.1 The path of the project in thirteen stages (J = research journal entries) 2

3.1 The hybrid sweep as a flow chart 11
3.2 The gradient-descent editor. Only the free numbers z_k are updated. 12
3.3 Why an additive edit was tried 13

4.1 Sorting 45 fact pairs by what GPT-2 Small already does with them 15
4.2 The early experiments 16
4.3 One run of the hybrid sweep (the Sophia prompt) 16
4.4 Effect of the layer of the intervention on the 12 eligible prompts 17
4.5 Making the computation fast 18
4.6 Final rank on 13 prompts, last token against all positions 18
4.7 How many features the first round can choose from 19
4.8 The safety-filter study 19
4.9 Repair and SAE-limit diagnostics on four failing prompts 20
4.10 Building the benchmark from CounterFact 21
4.11 Headroom on the 1,450 cached prompts 22
4.12 Learned strengths on the 300 test prompts 22
4.13 Gradient descent on six hand-picked prompts 23
4.14 Prompts brought to rank 1 by starting rank, 300 held-out prompts 24
4.15 Side effects and time, gradient descent against the wide sweep 24
4.16 Success against side effects 25
4.17 Control with random unrelated targets 26
4.18 What the edit does to other prompts 26
4.19 Generated text and instructions 27

5.1 ROME in the project 29
5.2 ROME and FeatureScalpel side by side 30
5.3 Causal trace of one prompt (Eiffel Tower) 32
5.4 Average causal trace of GPT-2 Small over 97 known facts 33
5.5 The two token positions compared 33
5.6 ROME's edit-layer sweep on GPT-2 Small 34

5.7 ROME and our edit scored with PS and NS 36

6.1 The two directions studied 39
6.2 The positive control, a country swap 40
6.3 How the effect depends on the dose 41
6.4 Effect on other prompts, Export through the ridge map 42
6.5 Top-1 of every export variant on the same 88 test records 42
6.6 More and more varied training edits did not help 43
6.7 Is the gain on reworded prompts specific to the fact? 44
6.8 Do the SAE's directions matter? 44
6.9 Import by mixing small's own state with medium's translated state 45

7.1 Software architecture 47
7.2 Layout of the Hybrid mute and boost tab 48
7.3 Layout of the Prototype lab tab 49

List of Tables

4.1 What the trial-and-error record kept and what it dropped. 27

5.1 ROME and FeatureScalpel compared. 31
5.2 The ROME baseline plan and where it stands [P15]. 31
5.3 ROME's scores and their counterparts in this project. 35

6.1 Export of the edit through the ridge map (88 primary test records; dose chosen on dev) [P16]. 41

8.1 Claims for the research paper and the evidence behind each. 52
A.1 Journal entries and where they appear in this report. 59
B.1 Pass bars ("gates") and their verdicts. 60
D.1 Figures, their data and the script that draws them. 61

Chapter 1

Introduction

1.1 Background and Motivation

Language models often finish a factual prompt with a generic word even though the right answer is only a few places down the ranking. Given "The location of Massachusetts Institute of Technology is in", GPT-2 Small [1] puts " the" first and " Cambridge" eighth [P3]. We call this **suppressed factual recall**. The name only describes the output ranking; it does not say what the model "knows". The usual fixes change the model itself. Fine-tuning and closed-form weight edits such as ROME [2] and MEMIT [3] are permanent, and when they are applied again and again they first wear away earlier edits and then break the model [4]. Retrieval and in-context editing [32, 29] leave the weights alone, but they hand the fact to the model as extra text.

Mechanistic interpretability gives a third option. A sparse autoencoder (SAE) [6, 7] rewrites a layer's activations in a much wider basis where only a few directions are active at a time. These directions, called *features*, often line up with single concepts. A feature is a direction in the residual stream [49], so we can turn it down (*mute*) or up (*boost*) during one forward pass and throw the change away afterwards. If that works, a wrong answer can be repaired **transiently**, with no weight touched, and the repair can be read as a short list of named features. This project asks whether it works, how to build it, and what it actually does.

1.2 How the Project Grew

The project, **FeatureScalpel** (first called Causal Feature Steering), began in July 2026 as a small prototype and grew through trial and error. We logged every step in a numbered research journal, which had 37 entries by October 2026, and this report follows that journal in order [P1, P17]. Figure 1.1 shows the whole path.

We want to be clear about how the pieces relate. The **main line** (stages 1 to 11) is the project itself: a transient SAE edit of GPT-2 Small, the question of how to choose that edit (a



Figure 1.1: The path of the project in thirteen stages (J = research journal entries). **How to read:** follow the boxes left to right and top to bottom. The outline colour says what the stage meant for the project: grey for the start, orange for a result that limited the approach, blue for a result we adopted, green for the line that was current at the time. Stages 1 to 11 are the main line (Chapters 3 and 4). Stage 12 is the check against ROME (Chapter 5). Stage 13 is an applied side study (Chapter 6).

hand-built search, a trained network or gradient descent), and what the edit does apart from the one number it is tuned for. The **ROME reference** (stage 12) is there so that our numbers do not rest on our own measures alone: it brings in the official implementation and the scores of the best-known editing method. The **cross-model transfer study** (stage 13) is not a later version of the system. It is an *applied side study*. Once the edit, the controls and the benchmark existed, we used them on a new problem to see whether the interpretability and intervention work was useful beyond the single model it was built for.

1.3 Problem Statement and Research Questions

Suppose GPT-2 Small ranks the true answer to a prompt below first. Can a transient edit of SAE features during one forward pass bring the answer to the top, how should that edit be chosen, and what does reaching the top tell us? Six research questions follow from this.

1. **RQ1 (what works).** Which way of choosing features and strengths brings a suppressed answer to rank 1, and which ideas fail?
2. **RQ2 (limits).** What holds such an edit back: the depth of the intervention, the safety rule, or the supply of features it can use?
3. **RQ3 (choosing the edit).** Can a program choose the edit better than a hand-built search, either a trained network or gradient descent on the edit itself?
4. **RQ4 (meaning of success).** Does rank 1 show that a fact was recovered? What else does the edit change: reworded prompts, neighbouring facts, unrelated prompts, generated text?
5. **RQ5 (outside check).** Are our measurements consistent with those of an established editing method, ROME, on the same model and with the same scores, and how does ROME behave on GPT-2 Small?
6. **RQ6 (application).** Can the change an edit makes in one model be carried into another model at inference time without changing weights, and what exactly is carried?

1.4 Objectives, Scope and Contributions

We set out to build a transient edit that changes only the intended feature contribution, to record with numbers what each design decision bought and which ideas failed, to build a benchmark with held-out prompts and exact paired statistics, to measure side effects against controls, to check our measures against ROME's, and to apply the tools to the transfer question with pass bars fixed before each run. We use GPT-2 Small with the SAE release `gpt2-small-res-jb` (24,576 features per layer) [15], layer 8 in all later studies, single-token facts from ROME's `known_1000` and CounterFact [2], and next-token measurements. The side study adds GPT-2 Medium, which has no SAE. Other models and a full evaluation of generated text are outside the scope. A head-to-head comparison with ROME, MEMIT or in-context editing on one record list is planned but has not been run (Section 5.8).

The main contributions are the following. (1) A record of the trial and error from single-feature ablation to gradient descent (Chapter 4) [P1, P13]. (2) A transient edit whose design choices are backed by measurements: delta patching, causal ranking, a strict per-feature safety filter, pool refill, and candidates from all prompt positions (10 of 13 prompts reach rank 1, against 4 with the last token only). A batched implementation cut model calls per prompt and layer from 186 to 6 with identical results [P5, P7]. (3) An equal-budget comparison. On 300 held-out CounterFact prompts gradient descent reached rank 1 on 271, against 161 for a sweep given the same range and 94 for the original settings, with lower side effects on the same prompt and 5 to 6.5 times less compute [P13]. (4) A control showing that rank 1 is a weak test: an additive edit pushes random words to rank 1 on 20 of 20 prompts. This led us

to withdraw an early "knowledge absent" classification [P14]. (5) A reference to ROME: the official implementation running on GPT-2 Small with causal tracing and an edit-layer sweep, a mapping between ROME's scores and ours, and a cross-check in which our edit scored with ROME's metrics generalises less than ROME's weight edit (reworded-prompt score 67% against 98.5%, on different records) (Chapter 5) [P15, P16]. (6) An applied side study with fixed pass bars: a linear translator between GPT-2 Small and Medium, Export and Import tests with controls, a documented search that locates the limit, and a first positive Import result (Chapter 6) [P16, P17].

1.5 Organisation of the Report

Chapter 2 reviews related work, including where gradient-based optimisation of edits comes from. Chapter 3 gives the mathematics of the final system, and Chapter 4 tells how we got there. Chapter 5 describes the ROME reference and the cross-check, and Chapter 6 covers the cross-model side study. Chapter 7 describes the application and the engineering, and Chapter 8 discusses the findings, the limits and the decisions that follow for the research paper. The appendices map journal entries to report sections, list the side study's pass bars, give the commands that reproduce the results, and trace every figure to the data it is drawn from. Results from our own experiments are cited to project records (labels P1 to P19 in the reference list), and statements about other work are cited to the literature.

Chapter 2

Literature Review

This chapter covers the work the project builds on, shows where our use of gradient descent sits in that literature, and states the gap we address. We read most sources through their abstracts and summaries rather than in full, and the numbers quoted are the sources' own.

2.1 Knowledge Editing

ROME [2] finds where a fact is stored in the middle MLP layers of GPT-style models and rewrites one weight matrix in closed form. MEMIT [3] extends this to thousands of edits. Both change the weights for good. Gupta *et al.* [4] showed that editing one fact after another slowly erases the earlier edits and then suddenly collapses the model, and Hase *et al.* [28] found that where a fact is stored does not necessarily tell you where it is best edited. ROME also gave us **CounterFact**, which pairs each prompt with a true and a counterfactual answer and adds reworded (paraphrase) and neighbouring (neighbourhood) prompts for testing generalisation and specificity [2, 61]. It also gave us the `known_1000` set of facts that its model already predicts, and like ROME we filter our facts in this way. Chapter 5 describes the method and its scores in detail. Some methods leave the weights alone and add the fact as text instead. In-context editing [29] puts the corrected fact and a few demonstrations in the prompt, and retrieval-augmented generation [32] conditions the model on retrieved documents. Both change the input and not the model's own computation.

2.2 Locating Facts and Measuring Edits

ROME rests on two ideas from earlier work. The first is that the feed-forward (MLP) layers of a transformer act as key-value memories, in which a key pattern in the input selects a stored value [56]. The second is causal mediation analysis, which asks how much of an effect passes through a given internal component [55, 54]. ROME applies it as *causal tracing*: corrupt the subject, restore clean states one place at a time, and see which restorations bring the answer

back [2]. Activation patching, the general technique behind this, is surveyed by Heimersheim and Nanda [19]. The residual stream, to which every layer adds its output, is the basic object of this view of the transformer [49].

Edits are measured on three questions [2]. *Efficacy* asks whether the new answer beats the old one on the edit prompt, *generalisation* whether it does so on reworded prompts, and *specificity* whether unrelated facts about other subjects are left alone. These become the Efficacy, Paraphrase and Neighbourhood scores that Chapter 5 defines, and the same three questions are used by later editing benchmarks and toolkits [3, 30]. We use them for the same reason as the authors: a change that works on the edit prompt alone, or that spoils neighbours, is not an edit worth having. Distinct-*n* counts [58] and a look at repetition loops [57] are the tools we use for the quality of generated text.

2.3 Activation Steering and Sparse Autoencoders

Activation steering adds a direction to the residual stream at inference time. Activation Addition [9] takes the difference between the activations of two prompts, contrastive activation addition [10] averages many such differences, and representation engineering [11] reads and controls directions that describe a whole population of examples. Superposition [5] means single neurons rarely match single concepts, so these directions tend to carry unrelated content with them. Sparse autoencoders [6, 7, 8] learn a wide basis in which only a few features are active at once, and those features are often interpretable. Open SAEs exist for GPT-2 Small [15] and Gemma [17], with browsable feature descriptions [18].

Several studies edit SAE features directly. Chalnev *et al.* [26] aim steering vectors at particular features to limit side effects. Zhao *et al.* [27] edit a few features to decide whether a model relies on its memory or on the context. Farrell *et al.* [25] unlearn knowledge by ablating features, and Marks *et al.* [31] use features as the nodes of editable circuits. Arad *et al.* [21] show that picking features by their effect on the output improves steering two to three times, which is the idea behind our causal ranking. The evidence is mixed, though. AxBench [12] reports that simple baselines can beat SAEs at steering, and a Google DeepMind update [22] reports negative SAE results on downstream tasks. Chaudhary and Geiger [23] find that SAEs struggle to reach a neuron baseline on factual knowledge in GPT-2 Small, and Cui *et al.* [24] show that behaviour suppressed by clamping SAE features can come back through the part of the activation the SAE does not explain. None of these tests exactly our setting. Together they are the reason we measure side effects and use controls, and why we assume nothing about what a feature edit does.

2.4 Causal Localisation and Attribution

Activation patching measures how much an internal component matters for the output [19]. Trying every component is expensive, so gradient-based shortcuts exist: attribution patching and AtP* [16] estimate component effects from a single backward pass. We rank features by actually removing them, in batches (Section 3.3), and keep the gradient-based version in mind for larger fact sets. The logit lens [34] reads an intermediate state through the output projection, and we use it in our diagnostics.

2.5 Gradient-Based Optimisation of Edits and Steering Vectors

We came to gradient descent through our own data (Section 4.10). It is an old tool in this area, though, and the work below shows where our use of it fits.

- **ROME itself.** Before ROME changes any weight, it finds the vector to be written by gradient descent on the probability of the new answer, with a term that keeps the subject's meaning intact [2]. Our loss has the same two ingredients, reaching the target and limiting the change elsewhere. The difference is that we tune a few hundred SAE multipliers for one prompt and change no weight.
- **Soft prompts and prefix tuning.** Lester *et al.* [42] and Li and Liang [43] freeze the model and train continuous vectors by gradient descent, which shows that a frozen model can be steered through a small optimised input.
- **Optimised steering vectors.** Subramani *et al.* [44] use gradient descent to find vectors that, added to GPT-2's hidden states, make it produce a chosen sentence almost perfectly. Dunefsky and Cohan [45] optimise a steering vector from a single example and study how far it generalises, including to harmful behaviour.
- **Adversarial examples.** Szegedy *et al.* [46] and Goodfellow *et al.* [47] showed that a small perturbation found by gradient descent can push a network to any output, and Zhang *et al.* [48] showed that networks can fit random labels. This is the right comparison for our additive edit (Section 4.11): once an optimiser has enough freedom, reaching the chosen output proves little.

As far as our limited search could tell, none of this work tunes one multiplier per *SAE feature* for a single prompt with a side-effect term, or compares that with a hand-built feature sweep on held-out prompts.

2.6 Comparing and Stitching Representations

Model stitching joins the lower layers of one network to the upper layers of another through a small learned layer. If the stitched network still works, the two representations are compatible [37, 38]. Similarity measures such as CKA [39] compare activations without stitching. For language models, Chen *et al.* [40] show that a simple affine map between the residual streams of two models moves features across at low cost. They carry SAEs from a smaller to a larger model, which cuts the larger SAE's training cost by about half, and they recover probes and steering vectors. Our translator (Chapter 6) is a ridge regression [41] of the same family, and we do not claim it as new.

2.7 Research Gap

Knowledge editing changes weights or the prompt [2, 3, 29], SAE steering works inside one model [21, 26, 27], and stitching work moves vectors learned in one model into another [40]. As far as a limited search could tell [P11], we did not find the following together: a joint mute-and-boost edit of causally ranked SAE features for a single fact, with a per-feature safety filter and pool refill; a comparison on held-out CounterFact prompts, with one yardstick, between a hand-built sweep and per-feature gradient descent; a control showing that an additive edit pushes unrelated words to rank 1; and a test of whether the change an edit makes in one model can be carried into another, with pass bars fixed beforehand. This report supplies these, negative results included. It does not claim that the SAE route beats prompting, probes, difference-of-means steering, ROME or in-context editing, because that comparison has not been made.

Chapter 3

The Edit and How It Works

This chapter gives the mathematics of the final system as it is implemented in `src/hooks.py`, `src/editing.py`, `src/hybrid_runner.py` and `src/gradient_editing.py`. Chapter 4 then shows how each piece came about.

3.1 Setting and Notation

GPT-2 Small reads a prompt and, at the last position, gives a probability for every possible next token. The **residual stream** is the running sum that every attention and MLP block adds its output to [49]. We write $x_p \in \mathbb{R}^{768}$ [1] for its value at position p at the hook point `blocks.L.hook_resid_pre`, with layer $L = 8$ unless stated otherwise. For a prompt whose true answer is t, the **rank** of t is its place in the next-token ordering (1 is first), and the **blocker** b is the token currently ranked first. An edit **succeeds** when t becomes first. That is a low bar (Section 4.11), so we always read it together with side-effect measures and controls.

3.2 SAE Features and Delta Patching

For the activation x_p the SAE gives sparse feature activations and a reconstruction,

$$f_p = \mathrm{ReLU}\big(W_{\mathrm{enc}}(x_p - b_{\mathrm{dec}}) + b_{\mathrm{enc}}\big) \in \mathbb{R}^{24576}, \qquad \hat{x}_p = W_{\mathrm{dec}}^{\top} f_p + b_{\mathrm{dec}}, \tag{3.1}$$

where row $W_{\mathrm{dec}}[k]$ is the direction of feature k in the residual stream. If we simply replaced x_p by $\hat{x}_p$ we would inject the SAE's reconstruction error (12 to 26% of the activation norm) and disturb the model even with no edit. So we add only the change in the decoded output. If feature k is scaled by s_k, with $s_k = 1$ for features left alone, then

$$\Delta_p = \sum_k (s_k - 1)\, f_{p,k}\, W_{\mathrm{dec}}[k], \qquad x'_p = x_p + \Delta_p, \tag{3.2}$$

at every prompt position p. Muting with strength $\mu \in [0,1]$ means $s_k = 1-\mu$, so $\mu = 1$ removes the feature, and boosting with strength $\beta \geq 0$ means $s_k = 1+\beta$. When every $s_k = 1$ the edit is exactly zero. The formulation was fixed in journal 12 [P5].

3.3 Choosing Candidates: Causal Ranking

Ranking all 24,576 features by removing each one would take as many forward passes. We do it in two stages instead. **Stage 1** screens. One clean pass caches the residual stream, the SAE encodes it, and we keep the top N features by activation, either at the last token or over *all* prompt positions except the start token, scoring each feature by its maximum activation. **Stage 2** ranks by cause. Each candidate is removed on its own and we record how a token's probability changes,

$$\Delta p_k(v) = P_{\text{ablate}(k)}(v) - P_{\text{clean}}(v). \tag{3.3}$$

For the blocker ($v = b$) the most negative values are its strongest drivers, which form the *mute list*. For the target ($v = t$) the most negative values are its strongest helpers, which form the *boost list*. The N removals are stacked into the batch dimension of a single model call, so ranking costs two batched calls instead of $2N+1$ separate ones [P5].

3.4 The Hybrid Mute and Boost Sweep

The **safety filter** tests every candidate alone at its strength and keeps it only if the target does not get worse:

$$\text{safe}(k) \iff r_k(t) \leq r_0(t) \ \wedge \ p_k(t) - p_0(t) \geq -10^{-6}, \tag{3.4}$$

where r and p are the target's rank and probability with feature k applied alone, and r_0, p_0 are the baseline values. A feature that is safe on both sides is kept on one side only. The sweep then applies the first k mutes and the first k boosts together in one forward pass, for $k = 1, 2, \ldots$, and stops when the target is first. It reports the best step and not the last one, because adding features that are each safe does not always help. When both pools are used up, **pool refill** keeps every feature applied so far, makes one pass to get the *steered baseline* and its new blocker, and ranks and filters the remaining features again, including ones rejected earlier [P3, P6]. Figure 3.1 gives the whole loop.

3.5 The Gradient-Descent Editor

The sweep decides which features to use and how many, with one strength per side. The gradient-descent editor instead gives each of $K = 200$ candidate features its own multiplier and tunes all of them for the prompt at hand. With free numbers z_k, set at the start so that no

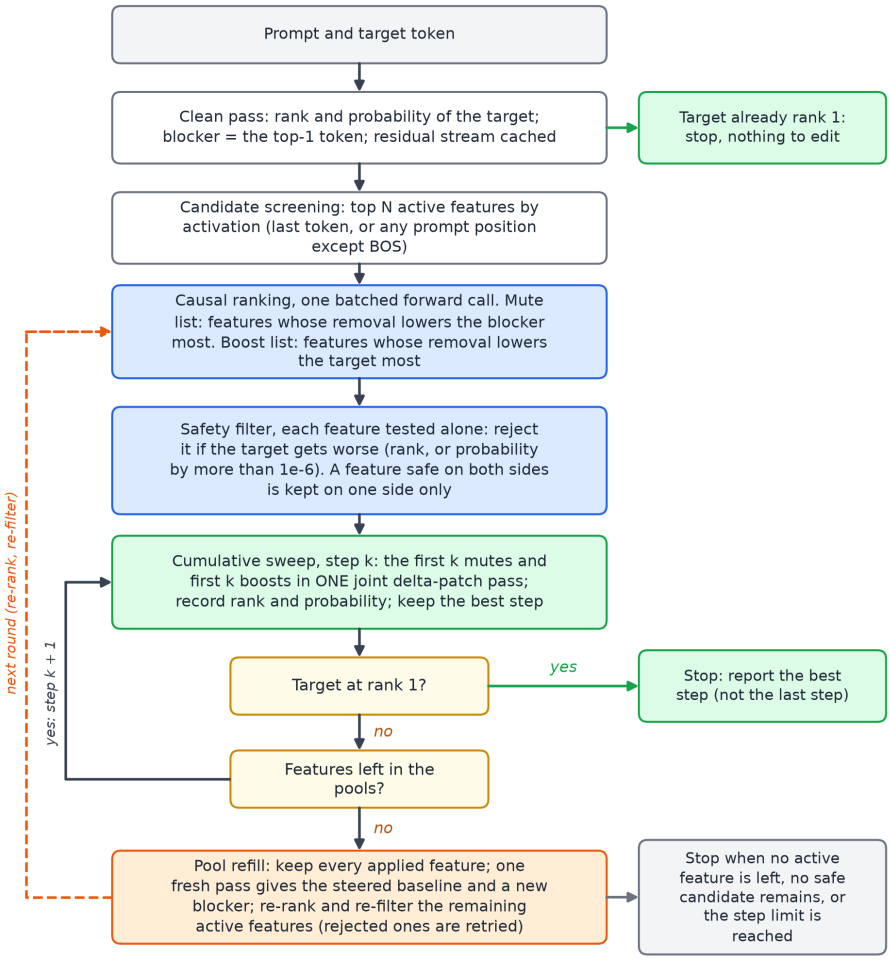


Figure 3.1: The hybrid sweep as a flow chart [P6]. **How to read:** follow the arrows from the top. Blue boxes are the ranking and filtering steps of Equations 3.3 and 3.4, the green box is the sweep itself, and the orange box is pool refill, which sends the loop back to the ranking step when the pools run dry. The loop stops when the target is first, or when no feature is left.

feature changes, the multiplier is $m_k = 1 + a_k$ with

$$a_k = -1 + 3\,\sigma(z_k), \qquad m_k \in (0,3), \tag{3.5}$$

where σ is the logistic function. With $A_{p,k}$ the clean activation of feature k at position p, the edit is Equation 3.2,

$$\Delta_p = \sum_{k=1}^{K} a_k A_{p,k} W_{\text{dec}}[k]. \tag{3.6}$$

We run blocks 8 to 11 from the edited state and read the logits ℓ at the last position. The loss is

$$\mathscr{L} = \max\bigl(0,\ \gamma - (\ell_t - \max_{v \neq t} \ell_v)\bigr) + \lambda_{\text{KL}}\,\mathbf{1}[\text{target leads}]\,\text{KL} + \lambda_{\text{size}} \sum_k |a_k|\, w_k, \tag{3.7}$$

with margin $\gamma = 0.3$, $\lambda_{\mathrm{KL}} = 3$, $\lambda_{\mathrm{size}} = 0.005$, and w_k the feature's share of the total maximum activation. The first term asks the target to beat every other token by γ logits. The second penalises changes to the rest of the distribution, measured by the Kullback-Leibler divergence [50],

$$\mathrm{KL} = \sum_{v \notin \{t,b\}} q_v \log \frac{q_v}{\tilde{p}_v}, \qquad q = \mathrm{softmax}\big(\ell^{\mathrm{clean}}_{\setminus\{t,b\}}\big), \;\; \tilde{p} = \mathrm{softmax}\big(\ell_{\setminus\{t,b\}}\big), \tag{3.8}$$

taken over all tokens except the target and the blocker, so that the intended change is not penalised. It is charged only once the target leads, which matches the sweep's stopping rule. The third term prefers small edits. Only the z_k are updated, by Adam [33] (learning rate 0.1, 100 steps), and GPT-2's weights and the SAE are frozen and receive no gradient. We keep the best iterate and re-check it through the same hook as the sweep in a full forward pass, so the reported rank does not depend on the shortcut used during tuning [P13]. Figure 3.2 shows the flow.



Figure 3.2: The gradient-descent editor. Only the free numbers z_k are updated.

3.6 The Additive Edit

A multiplier cannot switch on a silent feature, because $0 \times m = 0$ (Figure 3.3). The additive extension adds an amount $c_k \geq 0$ of the decoder direction of selected silent features, at the last position only:

$$\text{new volume} = \text{old volume} \times m + c, \qquad \Delta_{\mathrm{last}} \mathrel{+}= \sum_{k \in \mathscr{S}} c_k W_{\mathrm{dec}}[k]. \tag{3.9}$$

The set $\mathscr{S}$ holds the 200 features that are exactly silent at the last position and have the largest positive first-order score $W_{\mathrm{dec}}[k] \cdot g$, where $g = \partial(\ell_t - \max_{v \neq t} \ell_v)/\partial x_{\mathrm{last}}$ is the gradient of the

target's margin with respect to the residual. Each amount is $c_k = \text{cap} \cdot (\sigma(s_k) - \sigma(s_0))$ for $s_k \geq s_0$ and 0 otherwise. Every amount starts at exactly zero, cap is the loudest feature on the real prompt positions, and an extra loss $0.005 \sum_k c_k / \text{cap}$ discourages large additions. The score is a first-order estimate of how the margin changes. That is why this edit writes the *output direction of the answer* and not a representation of a fact (Section 4.11).

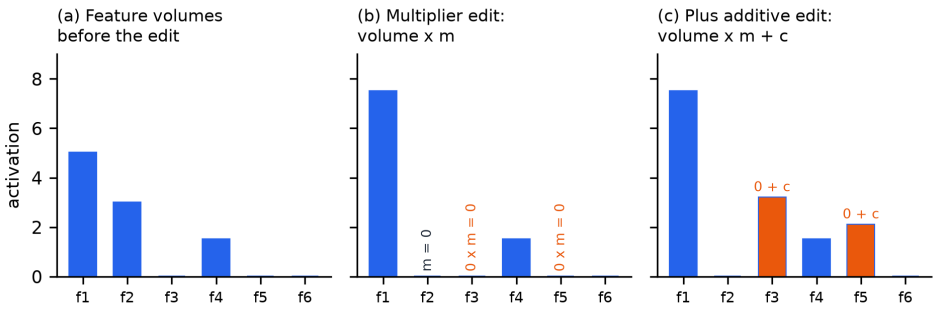


Figure 3.3: Why an additive edit was tried. A multiplier leaves a silent feature (f3, f5) at zero, while an additive amount c switches it on.

3.7 Benchmark and Metrics

The benchmark (16,360 CounterFact prompts, a subject-safe split, 300 test prompts) is described in Section 4.8. We report the *rank* and *probability* of the target; the *same-prompt KL* (Equation 3.8); the *edit size*, which is the norm of the change divided by the norm of the residual at the last position; and the effect of the same edit on 20 unrelated prompts, on CounterFact's reworded and neighbouring prompts, and on generated text. The reworded and neighbour scores are ROME's paraphrase and neighbourhood scores [2] (Table 5.3), which lets a reader compare them with the editing literature. Our success criterion, rank 1 over the whole vocabulary, is stricter than ROME's efficacy score, which compares only the new and the true answer. Methods are compared on the same prompts with the exact McNemar test [35] on reaching rank 1 and the Wilcoxon signed-rank test [36] on KL, and bootstrap intervals [53] are used where a difference of two shares is reported. We made our choices on validation prompts and only read the test prompts.

Chapter 4

Evolution of the System: a Record of Trial and Error

This chapter tells how we arrived at the system described in Chapter 3. For each stage we say what we tried, what happened and why we moved on, following the order of the research journal (Appendix A maps journal entries to sections). Every result is cited to the project record it comes from (labels P1 to P19 in the reference list), and every figure is drawn from stored data (Appendix D). The early experiments used one prompt or 22 facts. They explain why the design looks the way it does, but they are not performance estimates.

4.1 The Starting Point

The first prototype (July 2026, journal 1) loaded GPT-2 Small [1] and the layer-8 SAE [15] and added a *causal selector*. It took the features active on the last token of the prompt, removed each one in turn, and ranked them by how much the target's probability changed. Three things we learned there shaped the rest of the work [P1]. The most active feature is not the one that changes the answer, so we rank by causal effect and not by activation. Removing a feature completely damaged other predictions, whereas a partial change (strength 0.2 to 0.3) did not on the one example we tried. And an *eligibility rule*, applied to 45 fact pairs from ROME's `known_1000` file [2, 61], sorted the facts into three groups (Figure 4.1). The roadmap we wrote then already listed a learned predictor of which features to edit, which we reached in Section 4.9, and a comparison with ROME, which Chapter 5 describes and which is only half done.

4.2 Single-Feature Ablation

Our first search ablated, one after another, the features that *help* the target (strength 0.3, up to five rounds) on the 22 suppressed facts (journal 2). It corrected **1 of 22**. The reason is simple

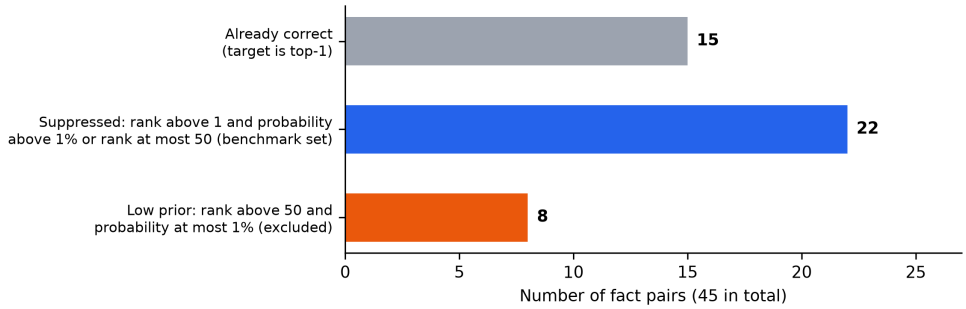


Figure 4.1: Sorting 45 fact pairs by what GPT-2 Small already does with them [P1]. **How to read:** each bar counts fact pairs. Grey: the true answer is already the model's first choice (15). Blue: it is not first but still receives probability above 1% or ranks within the top 50, so an edit has something to work with (22, the *suppressed* group used in the first experiments). Orange: the answer is unlikely and the pair was set aside (8).

in hindsight: muting what helps the target weakens the target. The second version muted the drivers of the current *competitor*, the token ranked first. It corrected **3 of 22**, each in one round and with no change to 15 control prompts (journal 3). All three started at rank 2, though, and the method failed whenever the leader was a generic word such as " the" (Figure 4.2a) [P2]. One feature was not enough, and we wanted to know why.

4.3 Several Competitors, Spread-Out Support

For a probe prompt ("The location of Massachusetts Institute of Technology is in", target " Cambridge", rank 8, with seven words above it) we muted all the competitors in proportion to their probability (journals 4 to 8). The seven competitors collapsed onto just two features: 313 for " the", " a" and " an", and 21169 for " question", " jeopardy", " danger" and " doubt". The target stayed at rank 8. Raising the strength on feature 313 by 73% moved the target's probability only from 1.29% to 1.39%, so muting too weakly was not the problem. When we looked at each competitor's top ten features, the semantic competitors turned out to be spread over many of them, with the second feature 92 to 95% as influential as the first. Muting the top three features per competitor reached rank 6, and five diluted the effect back to rank 7 [P3]. This is what the superposition picture predicts: a concept is rarely held by one direction [5].

4.4 Mute and Boost Together

Adding the opposite move, boosting the features that support the target, changed things at once. Three mutes and three boosts brought the probe prompt to **rank 1** (journal 5; Figure 4.2b). A fourth boost pushed it back to rank 2. This was not a bug, since an isolated rerun gave the same 6.7357%. The fourth feature supports " the" more than " Cambridge", and because probabilities pass through a softmax, more editing is not always better [P3]. The

lesson was to report the *best* step and not the last. The hybrid edit with a safety filter (rank 2 on this prompt, safer but weaker) became our working algorithm.

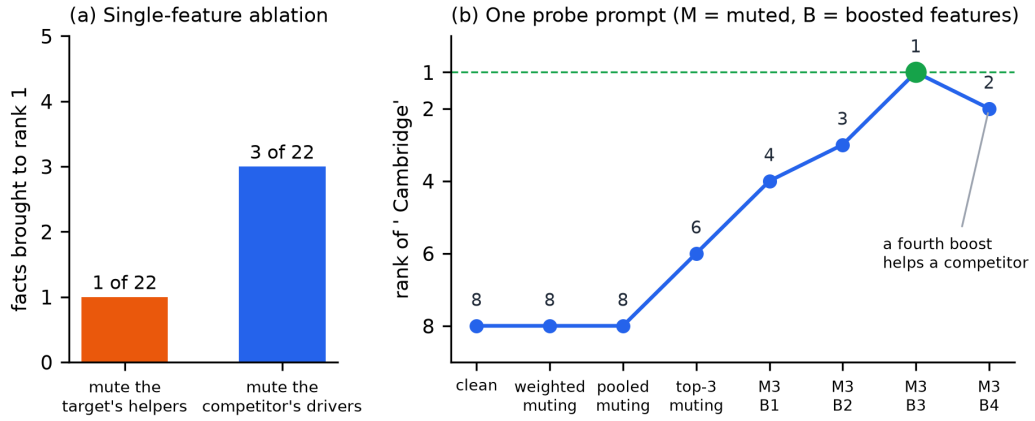


Figure 4.2: The early experiments [P2, P3]. **How to read:** (a) for 22 suppressed facts, how many ended at rank 1: muting the target's own helpers corrected 1, muting the competitor's drivers corrected 3. (b) For the probe prompt, the rank of "Cambridge" under three edits (lower is better): muting alone stayed at rank 8, mute and boost reached rank 1, and a fourth boost undid it.

The sweep applies one more mute and one more boost at every step, so its progress can be watched. Figure 4.3 shows a run on a prompt that needed 34 steps, from rank 6 to rank 1. The rank improves in jumps while the probability rises almost steadily, and rank 1 appears only at the very last step with 68 features changed. This is why we later asked where the sweep's supply of features comes from (Section 4.6) [P7].

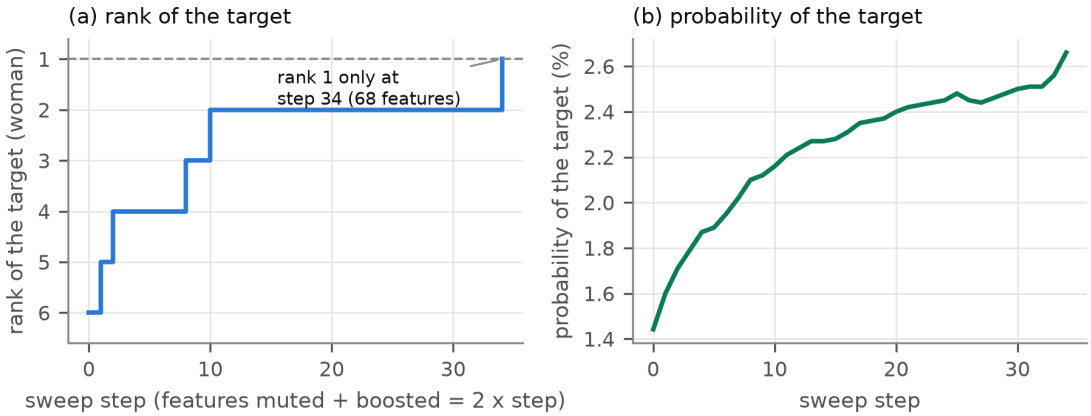


Figure 4.3: One run of the hybrid sweep on "This is Sophia, she is a", target " woman" (all prompt positions, layer 8) [P7]. **How to read:** the horizontal axis is the sweep step. (a) The target's rank (1 is first place, so the line should go up). (b) The target's probability. Each step mutes one more competitor feature and boosts one more helper.

4.5 Where to Edit, and Making It Fast

The SAE release we use has one dictionary for the residual stream before each of the twelve blocks and none for other hook points [15, P4], so the layer is the main choice. A layer sweep on 21 prompts (12 of them eligible) showed a plateau at layers 6 to 9, with layer 8 highest (Figure 4.4). Its mean rank gain was 7.33, against 7.08, 6.50 and 6.33 at layers 7, 6 and 9, and at most 4.75 at the other layers (journals 10 and 11) [P4]. We kept layer 8 as a reasonable default, not as a proven optimum.

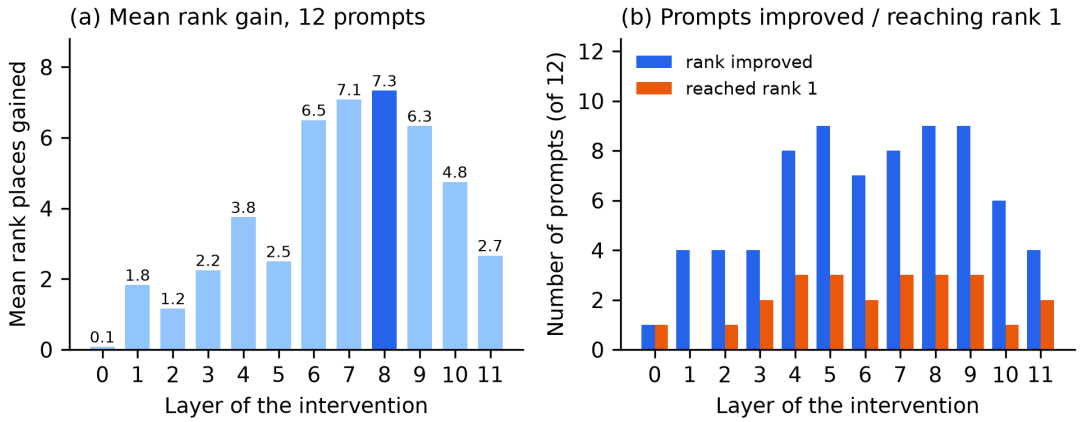


Figure 4.4: Effect of the layer of the intervention on the 12 eligible prompts [P4]. **How to read:** (a) mean number of rank places gained by the target at each layer (taller is better). (b) Number of the 12 prompts whose rank improved (blue) and whose target reached rank 1 (orange). The dark bar marks layer 8, the layer used in later work. The gain is high from layers 6 to 9 and low before and after.

The first real engineering step was a cost audit. A single prompt at one layer needed **186 model calls**. Both safety filters were recomputing the clean baseline inside every candidate loop, and passing it in removed 60 calls with identical results. Stacking the candidates into the batch dimension of one call then brought the number down to **6**, a 21-fold reduction. The ordered feature lists, the probabilities (to 10^{-9}) and all 60 safety decisions matched the sequential code, which we kept as the reference. A CUDA GPU gave another factor of 3.8 [20], and in the application the whole Hybrid run fell from about 12 s to under 1 s (Figure 4.5) [P5].

4.6 Making the Sweep Reliable

Once the sweep was fast, its weaknesses became visible (journals 16 to 19). It reported the last step instead of the best one, which we fixed with best-so-far tracking. Applying mute and boost as two chained hooks gave rank 66 where the combined safety check gave 59, so we switched to a single combined hook. And it stopped early, because its pool held only features that had passed the filter against the *clean* model. **Pool refill** fixed that by re-ranking against

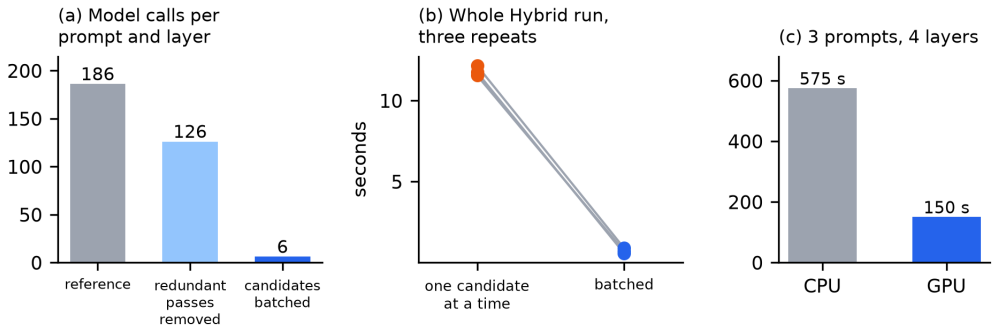


Figure 4.5: Making the computation fast [P5]. **How to read:** lower is better in all three panels. (a) Model calls needed per prompt and layer. (b) Whole Hybrid run with the same result, three repeats. (c) CPU against GPU on 3 prompts and 4 layers.

the edited model, and it exposed a hard limit. For "The leading cause of death in men is" only 67 features were active at the last token, and scaling cannot activate a feature that is off. The run used all 67 and stopped at rank 51 (from 152) [P6].

The fix was to draw candidates from **all prompt positions**, since the meaning of "Sophia" sits on the word *Sophia* and not on the last word of the prompt. On 13 comparable prompts the all-positions version reached rank 1 on **10**, against 4 for the last token. It won on 9 prompts and lost on none (exact sign test [52], $p \approx 0.004$), and in all 9 prompts where the last-token version failed, it had used every feature it could see (Figures 4.6 and 4.7) [P7]. On the 14 prompts that needed an edit, the all-positions version also starts with a pool about five times larger (a median of 276 candidate features against 54).

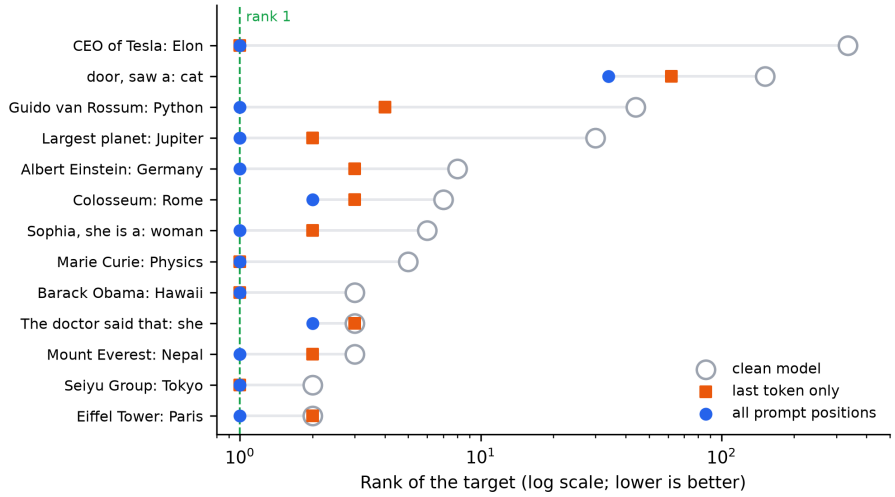


Figure 4.6: Final rank of the target on the 13 comparable prompts when candidates come from the last token only or from all prompt positions (log scale; rank 1 is the dashed line) [P7]. **How to read:** each pair of points is one prompt. A point lower on the page is a better result.

We then asked whether the strict safety rule was too strict. In a 45-prompt pilot, removing the filter lost 10 of 29 successes and gained none, a 5% tolerance lost 2, and graded strengths

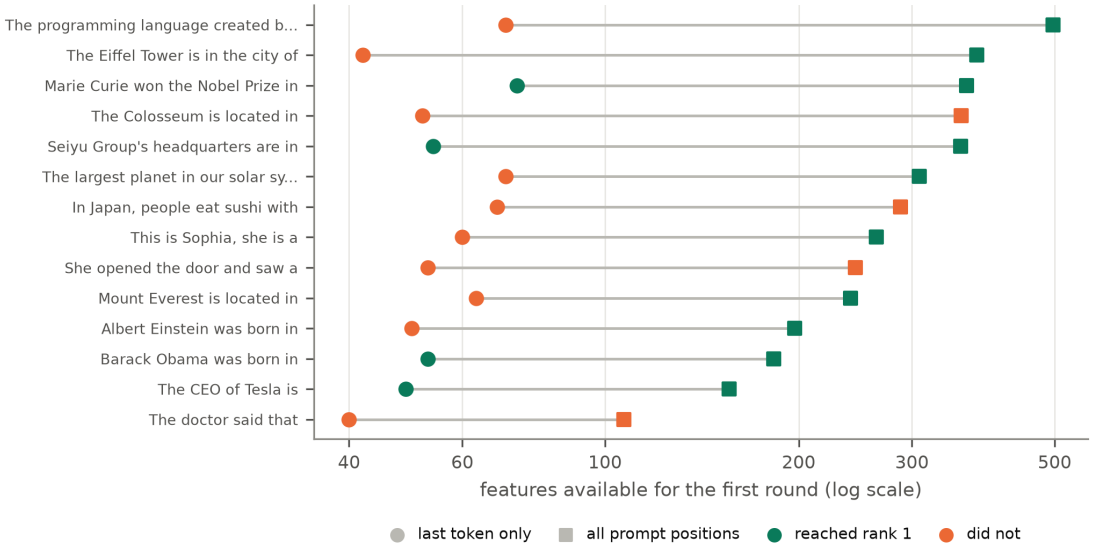


Figure 4.7: How many features the first round can choose from [P7]. **How to read:** each row is a prompt. The circle is the pool when only the last token is used, the square the pool when all prompt positions are used. Green means the target reached rank 1, orange means it did not. The all-positions pool is larger on every prompt, and the last-token pool is small on all of them. Prompts that were already correct are not shown.

changed nothing. On 131 last-token prompts, relaxed rules added at most one prompt (Figure 4.8). Only 2.5 to 2.9% of features were rejected on both sides, so the supply of features, and not the filter, was the real limit, and the strict filter stayed [P8].

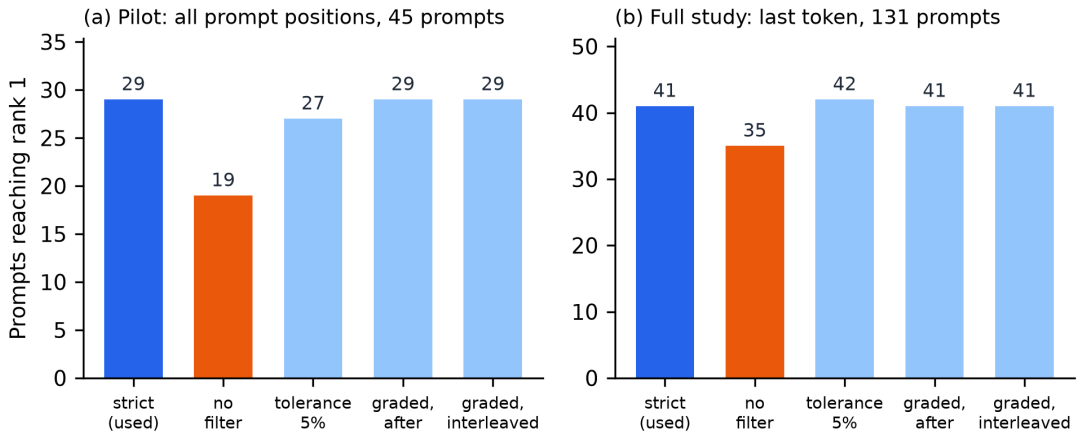


Figure 4.8: The safety-filter study [P8]. **How to read:** each bar is a version of the filter and its height the number of prompts brought to rank 1. (a) 45 prompts, candidates from all positions. (b) 131 prompts, last token only. "Strict" is the rule of Equation 3.4 and is the one we use. Switching the filter off (orange) loses prompts, and relaxing it (light blue) changes at most one.

4.7 Is the Layer, or the SAE, the Limit?

We tested three worries on the prompts that had failed in every mode (journal 22). Later layers might rebuild what the edit removes. The SAE might be more machinery than a factual edit needs. And the SAE's reconstruction error, which we never edit, might hold the missing information. The pushed direction was still 0.79 to 1.29 of its injected size at the output, and re-adding whatever fraction had been lost gave the same rank in 21 of 21 layer and prompt cells, so **we saw no repair**. A free residual edit of the same size reached rank 1 in all 21 cells, against 1 for the SAE edit (Figure 4.9). We chose these prompts because the SAE edit had failed on them, so this is a yardstick and not a success rate. The tool for reading the intermediate layers is the logit lens [34]. A literature check the same week (journal 25) found mixed evidence on SAE steering [12, 22, 23, 24] and led us to compare every later result with controls [P9, P11].



Figure 4.9: Repair and SAE-limit diagnostics on four failing prompts [P9]. **How to read:** (a) rank of the target after an SAE edit at each layer (dots) and after a free residual edit of the same size (crosses); the axis is reversed, so higher on the page is better and the dashed line is rank 1. The crosses sit at rank 1 everywhere, so a direction that works exists. The SAE edit reaches rank 1 in only 2 of the 22 cells shown (Colosseum at layer 7, and the probe prompt at layer 8). (b) Size of the pushed direction at each block after the edit, relative to the size injected (1.0 means unchanged). It stays between 0.8 and 1.3 of its injected size, so later layers do not undo it.

4.8 Scaling Up: the CounterFact Benchmark

Our hand-assembled sets (22, 45 and 131 prompts) share templates and are too small for held-out tests, so we built a benchmark from CounterFact (journal 24) [2]. Of its 21,919 records, 16,360 have a single-token true answer that GPT-2 Small ranks between 2 and 1000. We split them so that a subject appears in only one set and five relations are held out, which gave 1,000

training, 150 validation and 300 test prompts (Figure 4.10). For 1,450 of these prompts we cached the layer-8 states, the candidate features, their removal effects and the safety filter's verdict at ten strengths, once and for all. A **headroom** measurement then asked how much room any choice of multipliers has. Free per-feature multipliers could bring the target to rank 1 on **80%** of the 1,450 prompts if side effects are ignored, and on 41% with a side-effect penalty, against 6 to 15% for crude fixed rules (Figure 4.11) [P10].

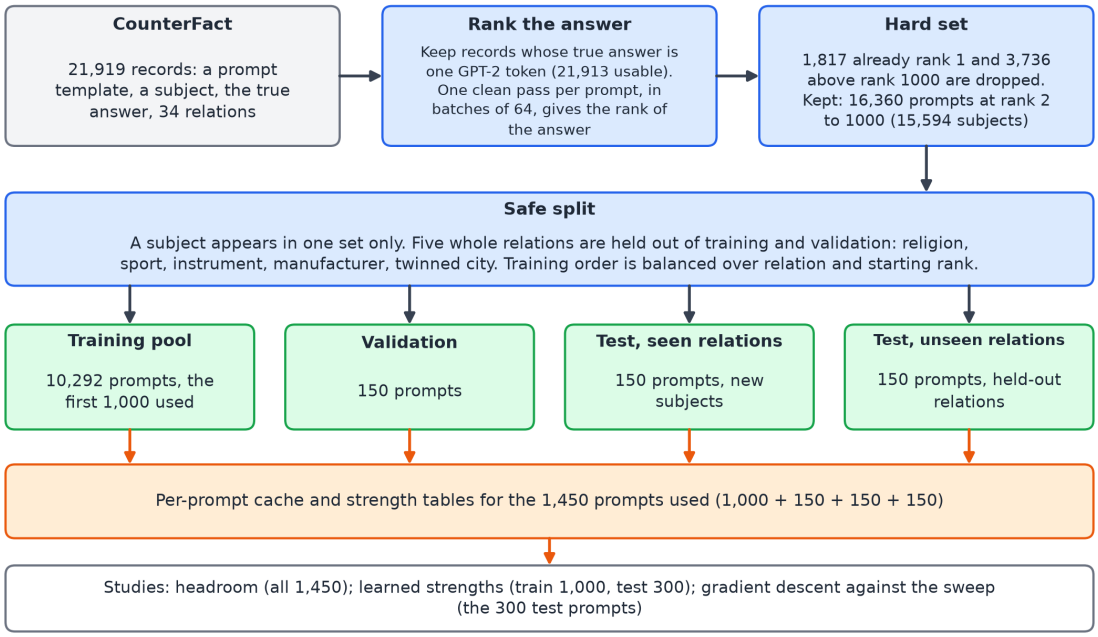


Figure 4.10: Building the benchmark from CounterFact [P10]. **How to read:** follow the arrows from the source data (top left) to the splits (green) and the studies that use them (bottom). Every subject appears in only one split, and five whole relations are kept out of training and validation.

4.9 Learning the Strengths

The sweep uses one mute strength (0.6) and one boost strength (0.5) for every prompt, even though the strengths change the outcome. We trained a small network to choose them (journals 23, 26 and 27). *PromptNet* reads 12 summary numbers about the prompt, such as the target's rank and probability, and outputs a mute and a boost strength. *FeatureNet* outputs one multiplier per candidate. Training back-propagated through blocks 8 to 11 from the cached states, with Adam [33]. We compared them with a *learned fixed pair* (two numbers shared by all prompts) and a *per-prompt oracle* tuned by gradient descent at test time. On the 300 test prompts, under a stand-in measure, fixed 0.6/0.5 reached rank 1 on 75 (25.0%), the learned pair (mute 0.98, boost 1.81) on 116 (38.7%), PromptNet on 119 and the oracle on 123 (Figure 4.12). The gain came from a better *global* pair. Choosing per prompt added only 3 prompts

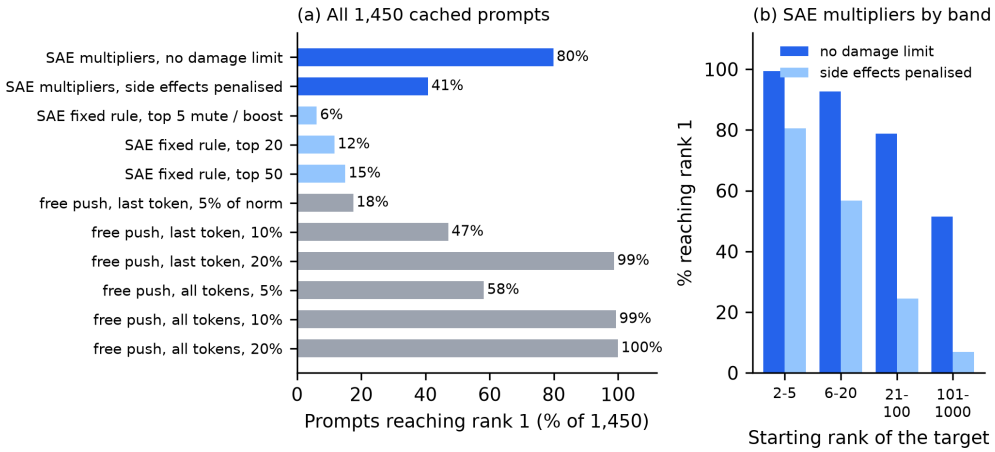


Figure 4.11: Headroom on the 1,450 cached prompts (one-shot edits on saved states, a stand-in for the real sweep) [P10]. **How to read:** (a) share of prompts reaching rank 1 for each way of choosing multipliers; the tallest bar is free multipliers without any side-effect penalty. (b) Free multipliers by how far down the true answer started.

(exact McNemar test [35], $p = 0.25$), side effects roughly doubled, and FeatureNet (31 to 32%) stayed below the pair. We never ran the real sweep with the learned strengths. The network turned out not to be the useful part. Its *teacher* was: free multipliers tuned on each prompt reached 73% in the same stand-in, nearly twice what the network managed [P12].

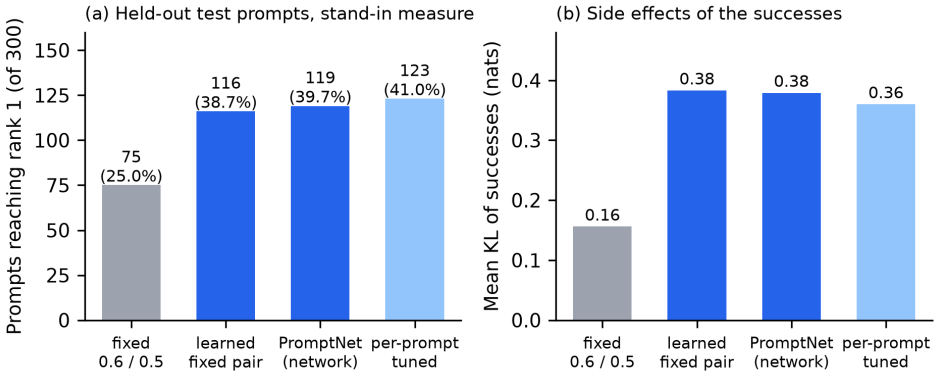


Figure 4.12: Learned strengths on the 300 test prompts (stand-in measure) [P12]. **How to read:** (a) prompts reaching rank 1 under each way of choosing strengths. (b) Mean KL of the prompts that succeeded, a measure of side effects (lower is better).

4.10 Gradient Descent Enters

Gradient descent was in the project for a week before it became the method. It first appeared as a measuring stick, as the bound for an edit made directly in the residual stream (journal 22, Adam on a free vector). It came back in the headroom measurement (journal 24) and in the oracle (journal 27). The oracle's 73%, against 31 to 32% for the network, suggested using

the optimiser itself as the editor (journal 28, October 3). It gives one multiplier per candidate, tuned for the prompt by 100 Adam steps, in about 6 to 8 s per prompt. Section 2.5 traces the idea in the literature. Figure 4.13 shows how it behaves on the first hand-picked prompts: the target's rank climbs steadily during the 100 steps, and the three prompts it solves there are ones where the target started within about 550 places [P13].

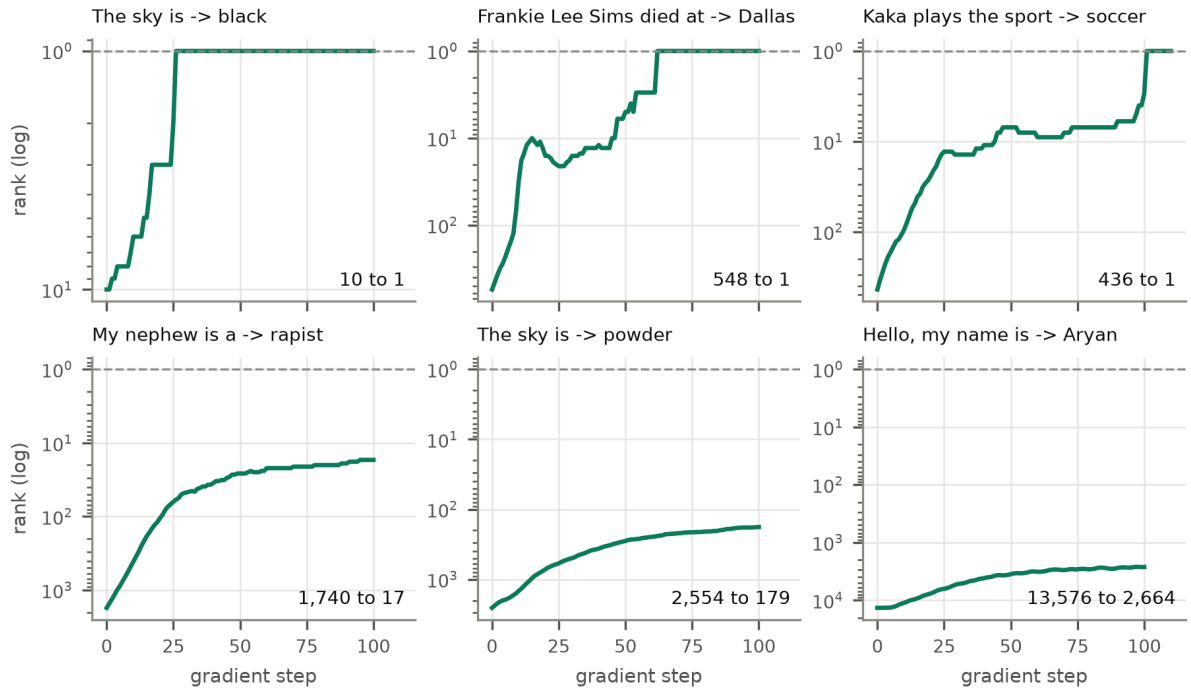


Figure 4.13: Gradient descent on six hand-picked prompts [P13]. **How to read:** each panel is one prompt and target. The line is the target's rank (log scale, so the line should climb toward the dashed line at rank 1) during the 100 tuning steps. The label at the bottom right gives the starting and final rank. Three prompts reach rank 1. The three others ("rapist", "powder" and a random name) improve a great deal but do not get there; their targets started at ranks 1,740 to 13,576.

The first hand trials favoured gradient descent but were unfair, because it could scale features from 0 to 3 times and the sweep only from 0.4 to 1.8. So the controlled comparison (journal 29) gave the sweep the same range (mute 1.0, boost 2.0) and measured all arms with one yardstick through the real hook. On the 300 test prompts gradient descent brought the true answer to rank 1 on **271** (90%), the wide sweep on 161 and the original sweep on 94 (Figure 4.14). It solved 110 prompts the wide sweep missed and lost none. Its same-prompt KL was lower on 160 of the 161 prompts that both solved, and it needed 7.7 s per prompt against 38.5 and 49.7 s (Figures 4.15 and 4.16) [P13]. Two cautions apply. Part of the KL advantage is built in, since its loss contains a KL term and the sweep has none. And the median target probability at rank 1 is only 7.7%.

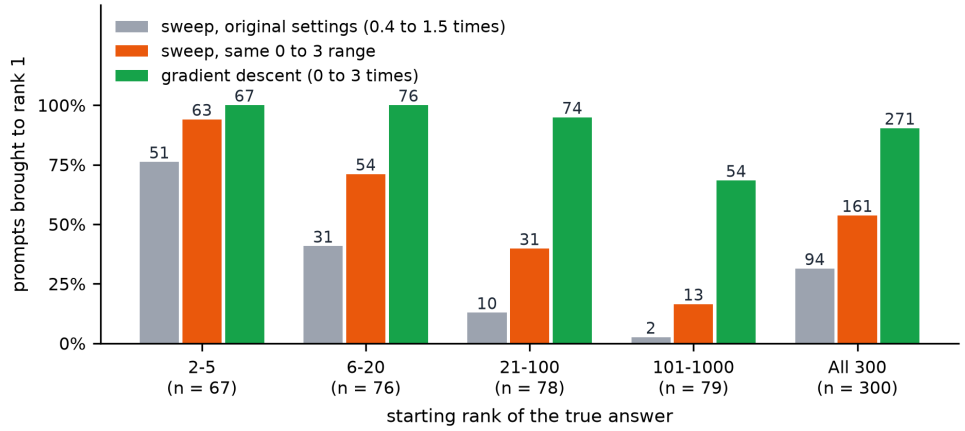


Figure 4.14: Prompts brought to rank 1 by starting rank of the true answer, 300 held-out prompts: the original sweep, the sweep with the same 0 to 3 range, and gradient descent (green) [P13]. **How to read:** the groups are bands of the true answer's starting rank, and the last group is all 300 prompts. Taller is better.

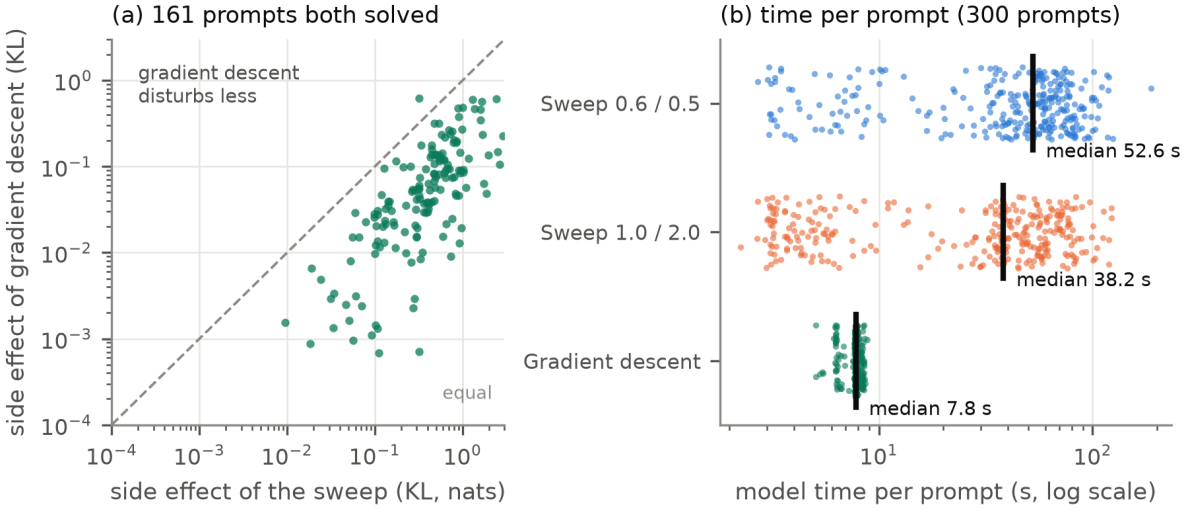


Figure 4.15: Side effects and time, gradient descent against the wide sweep [P13]. **How to read:** (a) each dot is one of the 161 prompts both methods solved. A dot below the dashed line means gradient descent disturbed the rest of the output distribution less (KL, lower is better). It is below the line for 160 of 161. (b) Each dot is one prompt's compute time for one method on a log scale, and the black bar is the median.

4.11 Forcing Every Target to Rank 1: the Additive Edit

The 29 prompts that gradient descent failed on are mostly deep targets whose helpful features may be silent, and a multiplier cannot switch on a silent feature. The additive edit of Section 3.6 can, and its first hand trials looked wonderful. All six hard prompts we tried reached rank 1, including absurd targets such as a random name after "Hello, my name is". We ran a control that paired 20 validation prompts with random, unrelated words. Multipliers reached rank 1 on 1 of 20, but the additive edit reached it on **20 of 20**, using an edit 126 to 192% the size of the residual stream itself (Figure 4.17). This is the analogue of fitting random labels

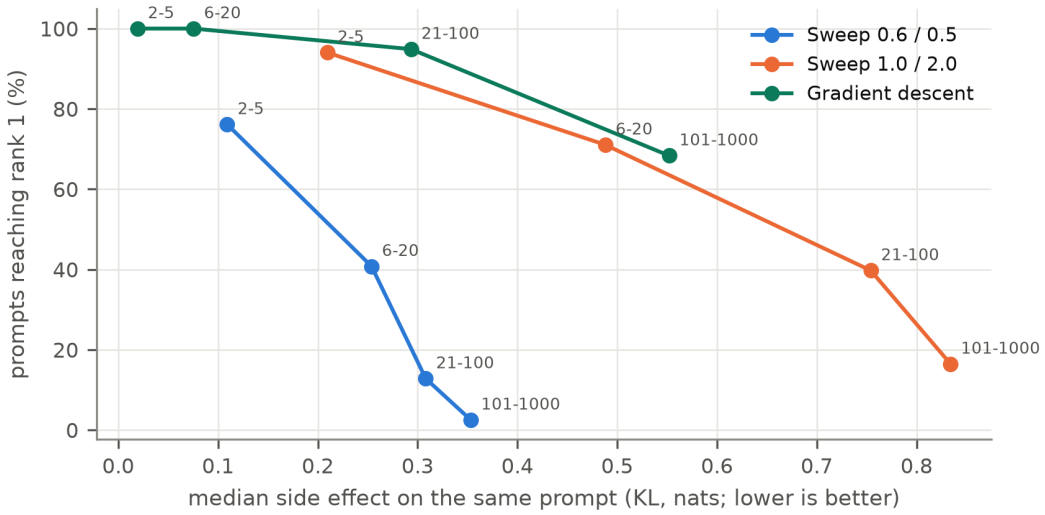


Figure 4.16: Success against side effects [P13]. **How to read:** each point is one band of starting ranks (labelled), for one method. Going up means more prompts solved, going left means smaller side effects, so the best methods sit in the upper left. Gradient descent (green) is above both sweeps in every band and to the left of the wide sweep in every band. In the hardest band the original sweep has the smaller side effect, but it solves only 3% of those prompts.

[48] or of an adversarial perturbation [46, 47]: with enough freedom, an optimiser reaches any output you ask for. The edit also carried over to reworded prompts about as well for a random word as for the true answer (median rank 4 against 4.5, and rank 1 on 25% against 16%), and it changed the top word on 82% of unrelated prompts. The edit *steers toward a word* and does not repair a fact [P14].

Three things followed. Rank 1 is a weak test, so every later result needs controls. Being able to steer a target to the top tells us nothing about knowledge in either direction, so we withdrew the early label "knowledge absent" as a statement about the model. And we switched the additive edit off by default and added a warning. It answered the question "can we force GPT-2 to say what we want?" with "almost anything", which is exactly why it is not a fix [P14].

4.12 What the Edit Does Beyond the Next Word

We then tested the multiplier edit on 40 test prompts (10 per band) for what it does besides reaching rank 1 (journal 30) [P14]. For *reworded prompts*, we tuned the edit on one prompt and applied it unchanged. It lowered the true answer's median rank from 94 to 36 and reached rank 1 on 9% of the rewordings, while a random word went from 17,504 to 11,832 and reached it on none. So there is a modest fact-related part, but the edit is mostly local (Figure 4.18). On *unrelated prompts* the multiplier edit changed the top word of 6% of 20 prompts, and the additive edit 17 to 24%.

For *generated text* (20 greedy tokens, edit kept on), fluency was unchanged (perplexity 4.4

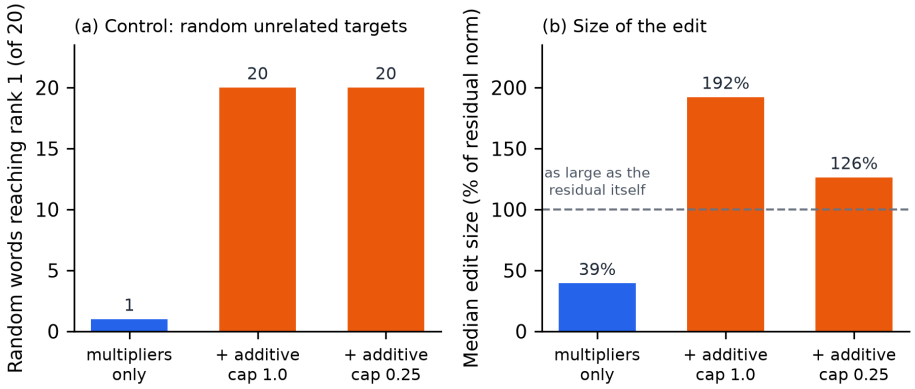


Figure 4.17: Control with random unrelated targets [P14]. **How to read:** each bar is the share of 20 validation prompts that reached rank 1 for a random word chosen as the target. A method that repaired facts should fail on random words. The additive edit reaches rank 1 on 20 of 20, at a size above the residual stream's, and multipliers on 1 of 20.

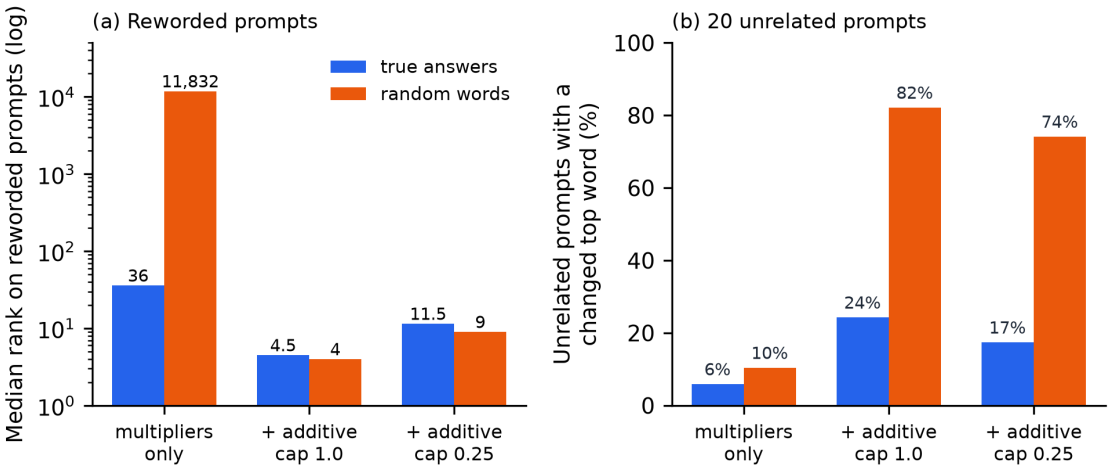


Figure 4.18: What the edit does to other prompts [P14]. **How to read:** blue is the true answer as target and orange a random word as target, for three kinds of edit. (a) The target's median rank on reworded prompts (log scale, lower is better). With multipliers the true answer reaches a median rank of 36 and a random word 11,832, whereas the additive edits bring both to between 4 and 12. (b) Share of 20 unrelated prompts whose top word changed (lower is better): the additive edits disturb unrelated prompts far more than multipliers do.

to 4.8 against 4.7), variety dropped slightly (distinct 2-grams [58] 0.85 to 0.89 against 0.93), loops, a known failure of repetitive generation [57], appeared in 2 to 12% of continuations against none, and the target word was repeated about 0.6 to 0.7 extra times. For *instructions*, we added the prefix "Do not say the word X". The unedited model already put X first on 45% of prompts, because a named word primes copying. The multiplier edit raised this to 68% and the additive edit to 88 to 90%, so the edit ignores the instruction (Figure 4.19) [P14].

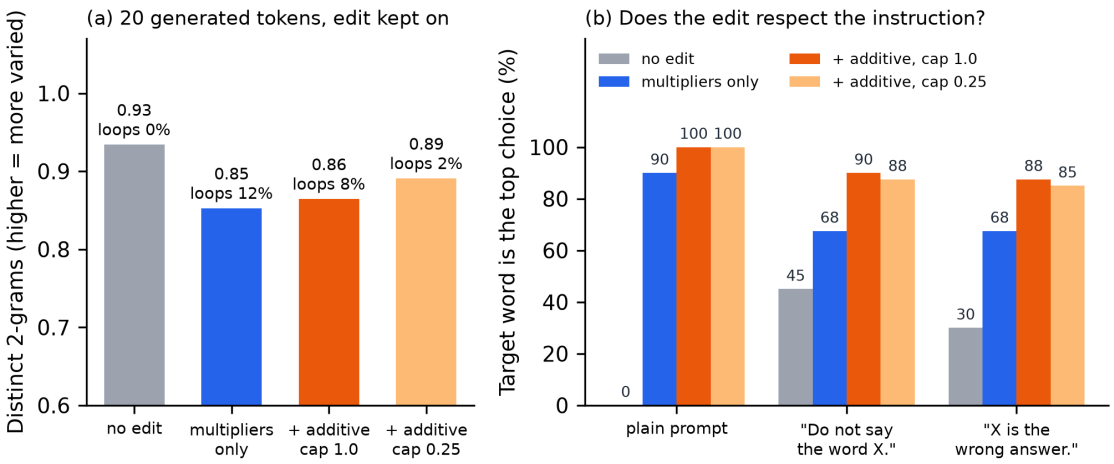


Figure 4.19: Generated text and instructions [P14]. **How to read:** (a) variety of 20 generated tokens (distinct 2-grams, higher means less repetition) and the share of continuations that fall into a loop, for the unedited model and three edits. (b) Share of prompts where the edit's target is the model's top word, for the plain prompt and for two prefixes that tell the model not to say it. Edited bars stay high with the instruction, so the edit overrides it.

Table 4.1: What the trial-and-error record kept and what it dropped.

Kept	**Dropped, and why**
Delta patching: reconstruction error never injected	Muting the target's helpers: weakens the target
Causal ranking by ablation, in batches	Weighted, summed or top-K muting alone: support is spread
Mute and boost with a strict per-feature filter	Relaxed or graded filters: at most one prompt gained
Best-step reporting, pool refill, candidates from all positions	Last-token candidates: features run out
Gradient-descent editor (271 of 300)	Learned strength network: no gain per prompt
Controls, held-out benchmark, fixed pass bars	Additive edit as a fix, and the "knowledge absent" label

4.13 Where the Main Line Stands

Table 4.1 sums up the record. A transient SAE edit brings a suppressed answer to the top on most held-out prompts, cheaply and with small side effects on the same prompt. It does this by pushing toward a word, it carries little beyond the prompt it was tuned on, and it ignores instructions. Three limits remained [P13, P14]. An edit is tied to one model that has an SAE, GPT-2 Small lacks answers that larger models hold, and rank 1 needs controls to mean anything. Two pieces of work followed from these. Chapter 5 sets the edit against ROME's own metrics, to check that our numbers are not private to our code. Chapter 6 asks whether the change an edit makes in one model can be carried into another.

Chapter 5

ROME as an External Reference

Every number in Chapters 3 and 4 was measured by us, with our own scripts, on our own benchmark. A reader can fairly ask whether those numbers are sound by any standard other than our own. This chapter describes the work that was done to answer that question using ROME [2], a widely used method for editing a single fact in a GPT-style model. It covers what ROME does, how it was set up in the project, what it showed on GPT-2 Small, and how its scores relate to ours. It also says plainly which part of the plan has not been carried out.

5.1 Why ROME Is in the Project

ROME serves two purposes, both written into the project roadmap as item B [P19].

1. **A shared yardstick.** ROME comes with the CounterFact data and with three scores that later editing work also uses [2, 3, 30]. If our edit can be scored with the same three numbers, a reader who knows the field can place our results without learning our private measures.
2. **A reference method.** ROME is an established editor. Running it on the same model and the same records would show where a transient SAE edit stands against a permanent weight edit, and would guard against the risk that our edits look good only because we compared them with weak alternatives.

The first purpose has been met. CounterFact and ROME's scores are built into our benchmark, ROME has been reproduced on GPT-2 Small with its official code, and our edit has been scored with ROME's own metrics (Section 5.7). The second has *not* been met. The head-to-head comparison on one record list, one scoring script and one machine was planned, and it has not been run (Section 5.8). Figure 5.1 shows what exists and what does not.

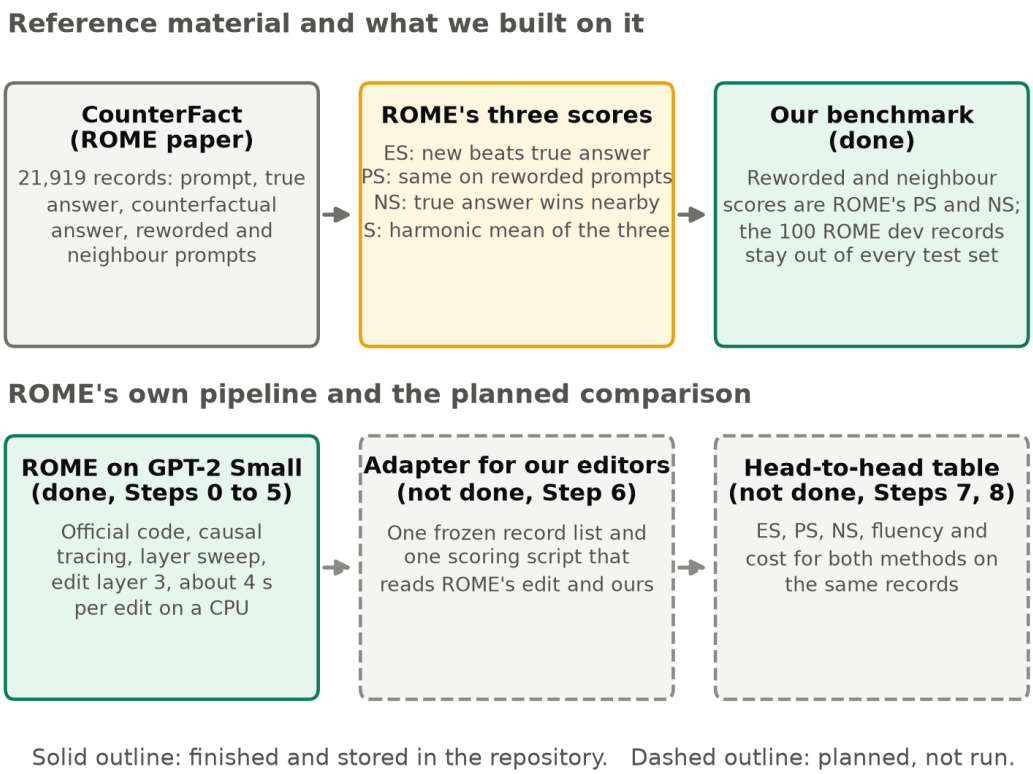


Figure 5.1: ROME in the project. **How to read:** the top row shows the data and scores we borrowed from the ROME paper and where they enter our work. The bottom row shows ROME's own pipeline on GPT-2 Small. Solid boxes are finished and stored in the repository, dashed boxes are planned and have not been run [P15, P19].

5.2 What ROME Does

ROME starts from the view that an MLP layer inside a transformer works like a key-value memory [56]. The first matrix of the layer turns the representation of the subject (for example "The Eiffel Tower") into a *key* k, and the second matrix W turns that key into a *value* that later layers read out as the attribute ("Paris"). To change the fact, ROME rewrites W so that the key of the subject produces a value that leads to the new attribute [2]. It takes four steps.

1. **Locate.** Causal tracing (Section 5.4) is used to find the layer and token where the fact is stored. For GPT-2 XL the authors report that mid-layer MLP modules at the last token of the subject matter most [2].

2. **Key.** The key k_* is the input to W at the last subject token, averaged over several short random prefixes so that it does not depend on one phrasing.

3. **Value.** Gradient descent finds a value v_* that makes the model say the new answer. The objective also contains a term that keeps the model's view of the subject unchanged, and a weight penalty [2].

4. **Update.** A closed-form, rank-one change of W writes v_* at k_* while disturbing other

keys as little as possible,

$$\hat{W} = W + \Lambda (C^{-1} k_*)^\top, \qquad \Lambda = \frac{v_* - W k_*}{(C^{-1} k_*)^\top k_*}, \tag{5.1}$$

where $C = \mathbb{E}[kk^\top]$ is the second moment of keys over ordinary text. The matrix C tells the update which directions are heavily used by everyday sentences and so should be left alone [2]. The change is permanent: every later prompt runs through the edited weights.

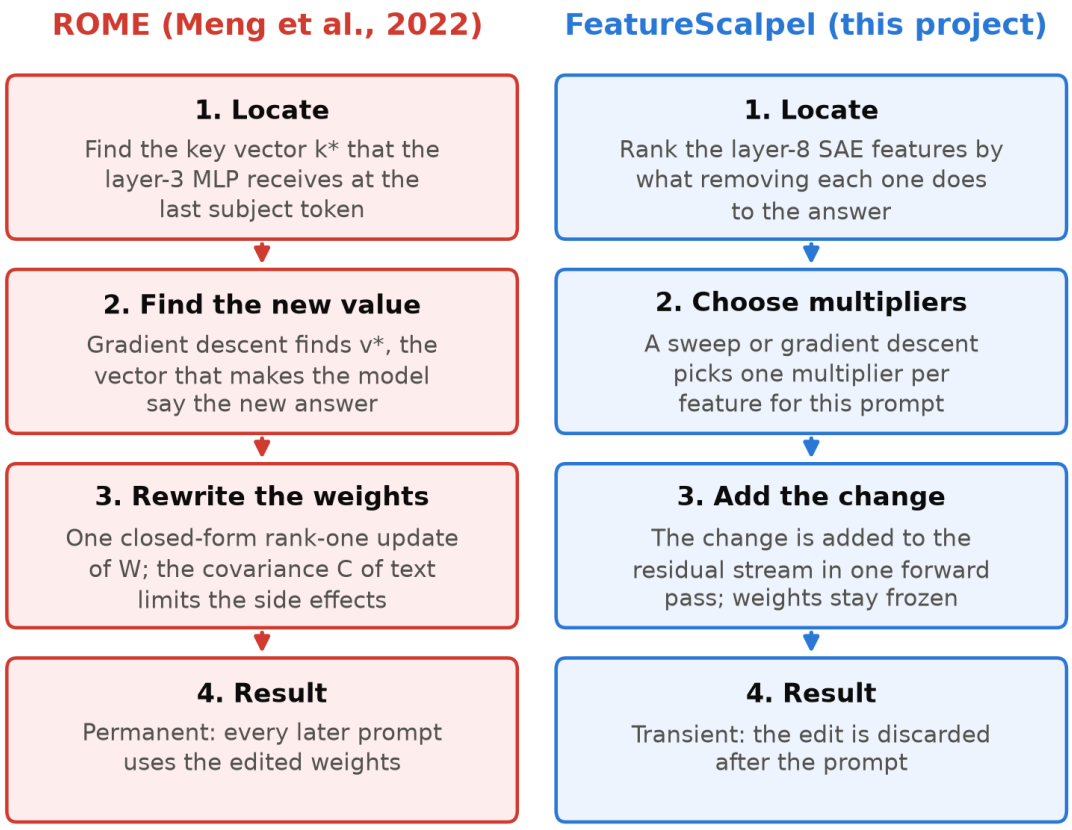


Figure 5.2: ROME and FeatureScalpel side by side. **How to read:** each column is one method, read from top to bottom. ROME changes a weight matrix once and for good. FeatureScalpel changes the activations during one forward pass and then discards the change. Both start by locating where the answer is decided [2, P13].

Table 5.1 sets the two methods against each other. The differences matter for any later comparison, because they mean that the two do not answer exactly the same question.

5.3 How ROME Was Set Up Here

A team member prepared the baseline on a separate branch (`rome-factual-editing`) on 7 October 2026 [P15]. The official code [60] was copied into `third_party/rome` at commit 0874014, and every local change is listed in a patch log. The changes are small: the device is selectable (CPU by default), baselines that we do not use were removed, GPT-2 was

Table 5.1: ROME and FeatureScalpel compared.

	ROME [2, P15]	**FeatureScalpel** [P13]
Where it acts	Weight matrix of one MLP layer (layer 3 in our GPT-2 Small runs)	Residual stream at layer 8, through SAE features
How long it lasts	Permanent	One forward pass
How the edit is found	Closed-form update after gradient descent for the value v_*	Hand-built sweep, or gradient descent on one multiplier per feature
What the edit is aimed at	A new answer for one subject and relation	A chosen word as the next token of one prompt
Side-effect control	Covariance C of ordinary text and a KL term	Safety filter, or KL and size terms in the loss
Cost per edit	About 4 s on a CPU, after one-time statistics over 20,000 text samples	7.7 s per prompt on a GTX 1660 Ti (gradient descent); the hardware differs, so the two are not comparable
What an edit looks like	A changed weight matrix	A short list of named features and multipliers

Table 5.2: The ROME baseline plan and where it stands [P15].

Step	**What it covers**	**Status**
0	Copy the official code, create the folder skeleton and the patch log	done
1	Make it run on a CPU; smoke test on GPT-2 Small	done
2a, 2b	Causal tracing of single prompts, then averaged over 97 known facts	done
3	Covariance statistics (Wikitext, 20,000 samples per layer)	computed, cache not in git
4	GPT-2 hyper-parameters and one worked edit (Eiffel Tower to Rome)	done
5	Edit-layer sweep on 100 development records	done, layer 3 chosen
6	Frozen benchmark, shared metrics and adapters	not done
7	Adapter for the hybrid editor, full comparison run	not done
8	Fluency measures, results table, plots, trade-off write-up	not done

added to the model choices, and the text source for the covariance statistics was switched to Wikitext [51] because the old dataset alias no longer resolves. The environment is Python 3.11 with a recent `transformers` release [59], and the covariance statistics are computed in 64-bit floats on the CPU. Table 5.2 lists the plan and its status.

The one worked edit shows that the pipeline is sound. Editing “The Eiffel Tower is located in the city of” to say “Rome” at layer 5 raised the probability of Rome to 0.998, the reworded prompts followed, and restoring the original weights recovered the original model exactly. In a second example, the Statue of Liberty, the probability of Rome rose only from 0.035 to 0.156, which shows that the strength of an edit varies from fact to fact [P15].

5.4 Causal Tracing on GPT-2 Small

Method. Causal tracing [2] is a form of causal mediation analysis [54, 55]. The subject's tokens are corrupted with noise (here three times the standard deviation of the token embeddings, about 0.39), which makes the model lose the answer. Then the clean internal state is put back at one position and layer at a time, and we record how much of the answer's probability returns. The place where restoring helps most is where the information that matters is carried. The residual stream is restored one layer at a time. For MLP and attention outputs a window of three layers is restored, following ROME's practice of restoring a window of layers for these two [2].

One prompt. Figure 5.3 shows the trace for one prompt. Almost all of the recovery is at the last token of the prompt in the upper layers. The subject tokens carry only a faint early effect in the MLP, much weaker than the mid-layer MLP effect at the last subject token that Meng et al. report for the larger GPT-2 XL [2]. A single prompt is a noisy guide: two further prompts (Space Needle and Windows Media Player) turned out to be facts that GPT-2 Small gets wrong, which leaves nothing to trace. So the trace was averaged.

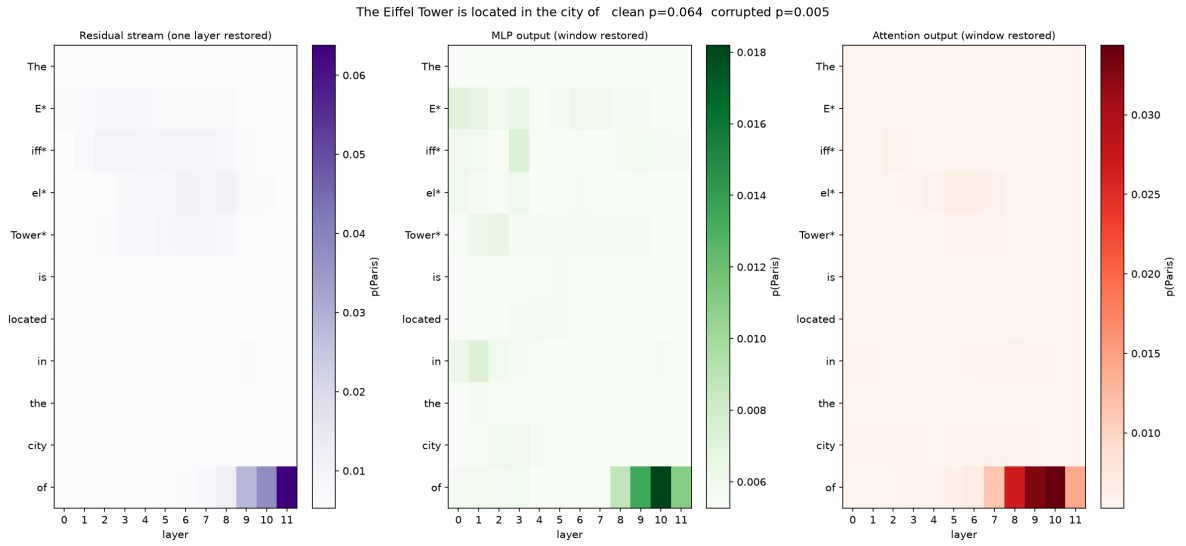


Figure 5.3: Causal trace of one prompt ("The Eiffel Tower is located in the city of", answer "Paris"), original plot of the baseline branch [P15]. **How to read:** each row is a token of the prompt (a star marks the corrupted subject tokens) and each column a layer. Dark cells are places where putting the clean state back brings the answer "Paris" back. The dark cells sit in the last row (the word "of") in the upper layers.

Averaged over 97 facts. We took the 342 prompts from ROME's known-facts file that GPT-2 Small completes correctly (out of 1,209) and traced the 97 whose clean probability for the answer was at least 0.2. The mean clean probability was 0.358, and under corruption it fell to 0.034, so the largest possible recovery is about 0.32 [P15]. Figure 5.4 shows the average and Figure 5.5 compares the two token positions that matter.

Reading. For GPT-2 Small there is no sharp mid-layer MLP site at the last subject token,

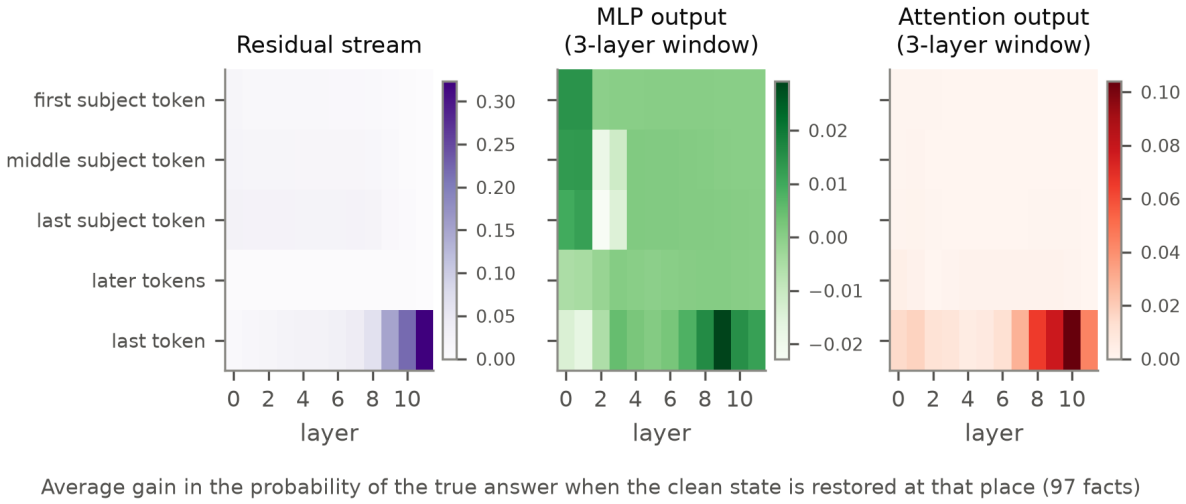


Figure 5.4: Average causal trace of GPT-2 Small over 97 known facts [P15]. **How to read:** rows are token roles and columns are layers, as in the previous figure. Darker means more of the answer's probability returns when the clean state is restored there. The strong cells are all in the bottom row (the last token) and in the upper layers.

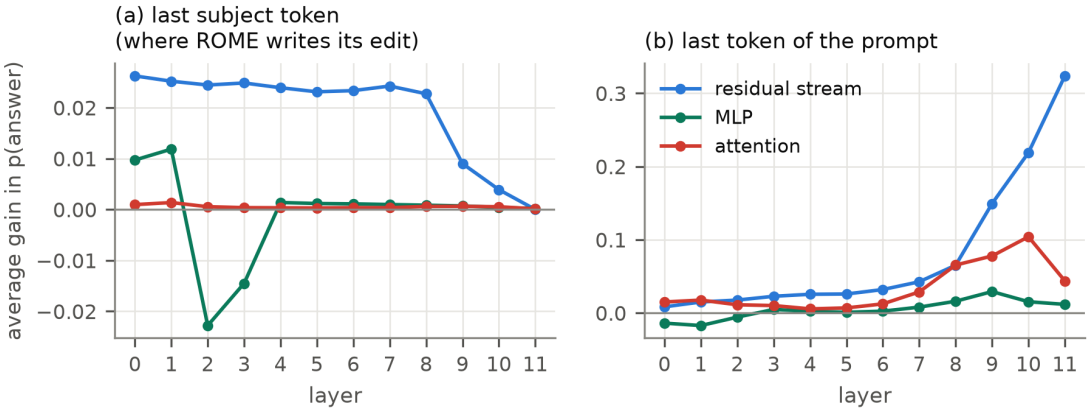


Figure 5.5: The two token positions compared [P15]. **How to read:** each line is one kind of internal state, and the height is the average recovery of the answer's probability. (a) At the last subject token, where ROME writes its edit, restoring the MLP at layers 0 and 1 helps a little (+0.010 and +0.012), restoring layers 2 and 3 hurts (around −0.02), and nothing else is above 0.001. (b) At the last token of the prompt, restoring the residual stream at layer 11 recovers 0.32 and attention at layers 8 to 10 adds up to 0.10.

so tracing does not point to one edit layer, and the layer for ROME was chosen by a sweep instead (next section). The strong, late effect at the last token is a separate matter. It says that the information that decides the answer is gathered into the last position in layers 8 to 11. Our own edit acts on the residual stream at layer 8, at every position, and our layer sweep found a plateau at layers 6 to 9 (Section 4.5). These two findings are consistent, but they come from different interventions (restoring a clean state, against scaling features), so we treat the tracing as supporting evidence for the range of layers and not as proof of the choice [P4, P15].

5.5 Choosing ROME's Edit Layer

Because tracing was not decisive, ROME was run at layers 1, 3, 5, 7 and 9 on 100 CounterFact records, drawn at random with seed 0 and kept apart from every later test set [P15]. The hyperparameters are ROME's, with the value-search settings listed in the patch log (20 gradient steps, learning rate 0.5, KL weight 0.0625, weight decay 0.5, and a limit on the size of the change of three times the original vector norm). The score compared is the harmonic mean S of the three scores defined in Section 5.6.

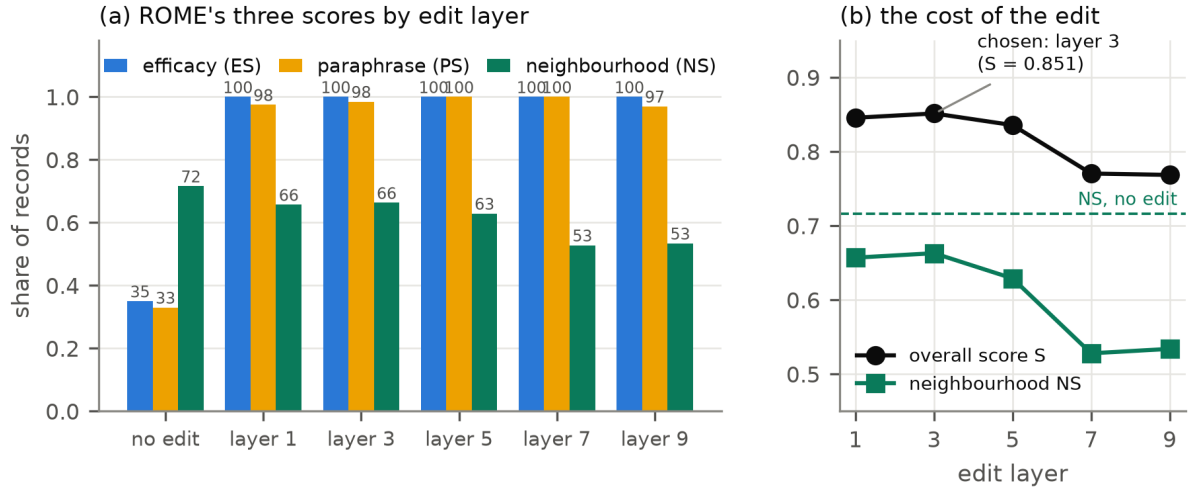


Figure 5.6: ROME's edit-layer sweep on GPT-2 Small, 100 development records [P15]. **How to read:** (a) for each layer, the share of records where the new answer beats the true one on the edit prompt (ES), on reworded prompts (PS), and where the true answer still wins on neighbour prompts (NS). The first group is the unedited model. (b) The overall score S and the neighbour score by layer. Efficacy is 100% at every layer, so layers differ only in the damage to neighbours.

Efficacy saturates, which makes the neighbour score decisive. Layers 1 to 5 are close (NS 0.66, 0.66 and 0.63) and layers 7 and 9 are clearly worse (0.53). Layer 3 has the best overall score ($S = 0.851$) and was chosen, but with 100 records the differences among layers 1 to 5 are within noise. Even at layer 3 the neighbour score falls from 0.716 to 0.663, about five points. The ROME paper reports its results for GPT-2 XL, a model more than ten times larger [1, 2], and we did not try to compare absolute values across model sizes. An edit took about 4 s on a CPU [P15].

5.6 How ROME's Scores Relate to Ours

ROME judges an edit with three scores, each a share of records [2]. **Efficacy (ES)** is the share where the model gives the new answer a higher probability than the true one on the edit prompt. **Paraphrase (PS)** is the same on reworded prompts. **Neighbourhood (NS)** is the share of prompts about *other* subjects with the same relation where the true answer still beats

Table 5.3: ROME's scores and their counterparts in this project.

ROME [2]	**What it asks**	**Our counterpart**
ES	New answer more likely than the true one on the edit prompt	Target at rank 1 of the whole vocabulary (stricter, Section 3.7)
PS	Same, on reworded prompts	"Rewordings" of Chapter 6, the same definition
NS	True answer still more likely than the new one on neighbour prompts	"Neighbours" of Chapter 6, the same definition; also KL and top-word flips on unrelated prompts
S	Harmonic mean of ES, PS and NS	Not used; we report the three separately

the new one, so it measures damage. The overall score *S* is their harmonic mean. In the official code each probability is the product over the answer's tokens [P15, 60].

Our benchmark was built to speak the same language. Table 5.3 gives the correspondence. The reworded and neighbour scores of the transfer study (Chapter 6) are ROME's PS and NS. Its records are drawn from CounterFact [2], each with two reworded prompts and five of its ten neighbour prompts, and the 100 ROME development records are excluded from every one of our test sets by drawing them with the same seed (0) and removing them [P16, P18]. There is one important difference. ES asks only whether the new answer beats the true answer, which is a comparison of two words. Our success criterion, rank 1, asks whether the target beats *every* word in the vocabulary. Rank 1 is therefore the stricter test, and an ES of 100% does not mean the new answer is the model's top choice.

5.7 A Cross-Check: Our Edit Scored with ROME's Metrics

The check asks whether the numbers we report look reasonable when read with ROME's yardstick. Two kinds of evidence exist.

Base rates. Three samples of facts were drawn for different reasons, and they should not agree exactly. They do agree in size order with what each sample was built to contain. On ROME's known-facts file, GPT-2 Small completes 342 of 1,209 prompts correctly (28%) [P15]. In the 45 fact pairs of our first eligibility test, which were drawn from the same file, 15 (33%) were already correct [P1]. On all CounterFact prompts with a one-token true answer, only 1,817 of 21,913 (8.3%) are already at rank 1 [P10], which is expected because CounterFact was not selected for what the model knows. We use the last figure when we say that the unedited Small model answers 8.3% of facts correctly (Section 6.6).

Our edit scored with PS and NS. In the controlled Export study the gradient-descent edit was tuned in GPT-2 Small toward each record's counterfactual answer, and the same multipliers were then applied to the reworded prompts and the neighbour prompts [P16]. This gives PS and NS for our own edit on 88 test records on which the edit reached rank 1. Figure 5.7

puts these next to ROME's layer-3 edit on its 100 development records.

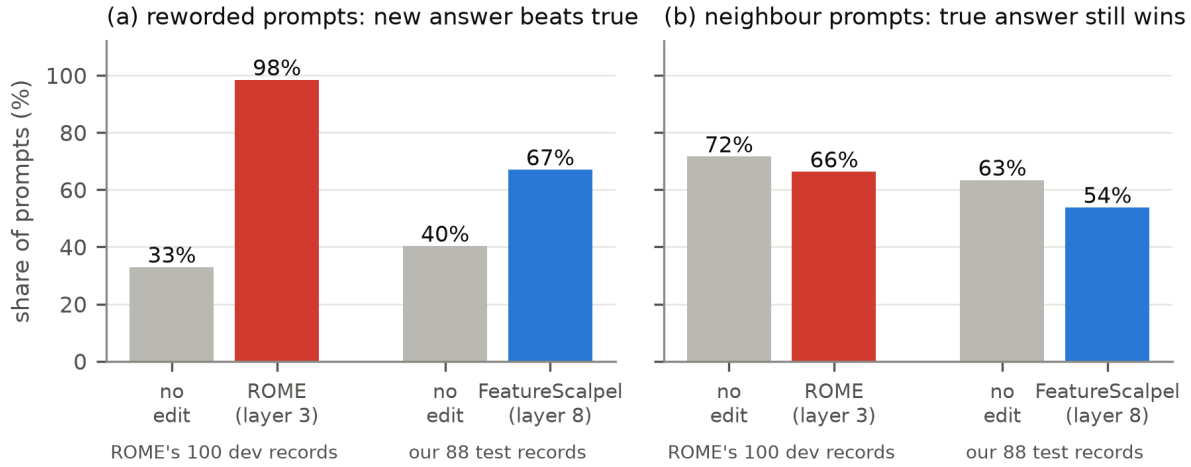


Figure 5.7: Reworded-prompt (PS) and neighbour (NS) scores for ROME (layer 3, 100 development records) and for our gradient-descent edit (layer 8, 88 test records), each with its own unedited baseline [P15, P16]. **How to read:** the grey bar is the unedited model and the coloured bar the edited one, on the record set named below each pair. Higher is better in both panels. The two record sets are different, so this is a rough comparison and not a controlled one.

By ROME's own yardstick the weight edit generalises further and does less damage [P15, P16]. ROME lifts the reworded-prompt score from 33% to 98.5% and loses 5 points of neighbour score (72% to 66%). Our edit lifts it from 40% to 67% and loses 9.5 points (63% to 54%). Two things should keep this from being over-read. The record sets differ: ROME's are a random sample, and ours are records where neither GPT-2 model already answers the counterfactual word and where the edit succeeded on the tuned prompt. And the two edits are made differently. ROME's changes the weights once, while the multipliers of ours are recomputed for each reworded text.

What the cross-check supports is modest but useful. The no-edit levels of both are in the same range (PS 33% and 40%, NS 72% and 63%), so our measurement of the same quantity is not out of line with the official code. And the gap in rewordings agrees with our own finding that the SAE-feature edit is mostly a push toward a word and carries little beyond the prompt it was tuned on (Section 4.12). The check does *not* show that ROME is better in general, since the records, the models' edit targets and the way each edit is re-applied all differ [P15, P16].

5.8 What Remains and What the Paper Needs

The comparison that the roadmap asked for is still open [P15, P19]. To make it a fair one, the following has to be fixed before anything is run, in line with the practice of Appendix B.

1. **One record list.** Freeze about 150 CounterFact records, disjoint from the 100 ROME development records, on which neither model already prefers the counterfactual answer.

2. **One scoring script.** Score every method with ES, PS and NS as ROME defines them, and also with the rank-1 criterion and the same-prompt KL of this report, so that the stricter and the standard measures can be read side by side. Add fluency and consistency only after the extra data files for those measures are set up.
3. **Controls and pass bars.** A random-word edit of matched starting rank, and a pass bar for each question written down before the run.
4. **Cost on one machine.** Time ROME and our editor on the same hardware, because the numbers in Table 5.1 come from different machines.
5. **Other uses of ROME.** ROME-edited copies of GPT-2 Medium were planned as a second donor for the Import study (Chapter 6). That has not been run.

Until this is done, the report makes one claim about ROME and no other: ROME runs on GPT-2 Small with the official code, and our edit scored with ROME's own metrics behaves like the push that the rest of our evidence describes. It does not say how the approach ranks against ROME, MEMIT or in-context editing.

Chapter 6

Applied Study: Carrying an Edit to Another Model

This chapter describes a *side study*. It is not the next version of the system from Chapters 3 and 4. We took the tools that work had produced (the edit, the benchmark, the controls and the habit of fixing pass bars first) and applied them to a new question, to see whether the interpretability and intervention work was useful beyond one model. The scores it uses for rewordings and neighbours are ROME's (Chapter 5). Results are cited to the project records [P16, P17, P18], and the pass bars are listed in Appendix B.

6.1 The Question and the Design

Can the change an edit makes inside GPT-2 Small be translated and added to GPT-2 Medium at inference time with no weight change, and can Medium's state improve Small? We call the first direction **Export** (small to medium) and the second **Import** (medium to small). Figure 6.1 shows both. The two models do not share internal coordinates (width 768 against 1024) [1], so we need a *translator*. And since Section 4.11 showed that "the answer reaches rank 1" is weak evidence, every test here has controls. The idea of moving representations between models by a learned map follows earlier work on model stitching [37, 38, 40].

Models and data. "Small" is GPT-2 Small with its layer-8 SAE, and "medium" is GPT-2 Medium (24 layers, width 1024, no SAE) [1]. They share one tokenizer. We pair small's layer 8 with medium's layer 16. The benchmark is 200 CounterFact records [2] from the hard set of Section 4.8, split into 50 development (dev) records and 150 test records. Dev records are used to choose settings (layer, dose, epoch), and test records are only read. The 100 development records of the ROME baseline are excluded. The edit tuned in small reaches rank 1 on 116 of the 200 records. The **88 primary test records** are the test records where it does, because only there is there an edit to carry over. Before anything is done, medium's median rank for the target is 142 [P16].

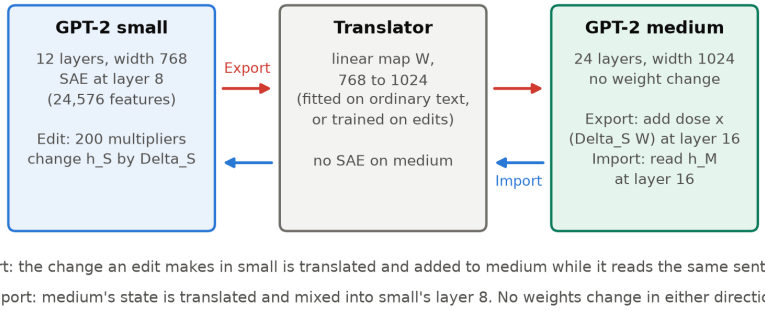


Figure 6.1: The two directions studied. **How to read:** the left box is GPT-2 Small, the right box GPT-2 Medium, and the middle box is the translator. Red arrows: Export carries the change an edit makes in Small into Medium. Blue arrows: Import carries Medium's state into Small. No weight changes, and the SAE is used only where the edit is made [P16].

The translator. We translate only the *change* an edit makes, so the bias of an affine map cancels and the translated change is $\Delta_S W$ (zero at the first position). We tried four ways of getting W. (1) A *ridge map* fitted on ordinary text, $W = \arg\min \|(H_S - \mu_S)W - (H_M - \mu_M)\|^2 + \lambda \|W\|^2$, over paired states of the two models on Wikitext-2 [51] (500,126 fit tokens), using ridge regression [41]. (2) A *neural map*, which is the ridge map plus a one-hidden-layer correction. (3) A *linear map trained on edits*, which starts from the ridge map and is trained so that medium's answers move the way small's do. Its goal is $q = \text{softmax}(\ell_M + \ell_S^{\text{edit}} - \ell_S)$ and its loss is $\text{KL}(q \,\|\, p_M(h_M + \alpha \Delta_S W))$ [50] over 289 successful training edits. (4) *No fixed map*: we re-tune the 200 multipliers for each sentence against *medium's* output. This is a ceiling and not a transfer, since it looks at medium every time.

Export, Import and controls. Export adds the translated change while medium reads a sentence, $h'_M = h_M + \alpha \Delta_S W$ at layer 16 at every position except the first. The **dose** α is chosen on dev. From step D10 on we chose it by the best mean log-rank gain, because choosing by dev top-1 on 28 records proved fragile. Import mixes small's state with medium's translated state, $h'_S = (1-\gamma)h_S + \gamma T(h_M)$, where $\gamma = 1$ means full replacement. The receiving model is never shown the answer. The controls are a random vector of the same size, another record's edit, an edit toward a random word of matched starting rank, a *word push* along the target's output direction, and the fact stated in medium's prompt. The word push uses the answer, so it is a comparison and not a method. For Import the controls are a state from a wrong sentence, the average state and a random map. *Rewordings* and *neighbours* are the share of reworded prompts on which the new answer beats the true one, and the share of other-subject prompts on which the true answer still wins (ROME's PS and NS, Table 5.3). **Every pass bar ("gate") was written into the project plan before the run that judged it, and none was moved.** A step designed after we saw a result is labelled post-hoc and cannot change a verdict (Appendix B) [P18].

6.2 The Channel Works

The ridge map fits well enough to keep the receiving model working. Per-dimension R^2 is 0.58 from small layer 8 to medium layer 16 and 0.66 from medium 16 to small 8. When we stitch one model's first layers to the other's last layers through the map, it recovers 91% (Export) and 98% (Import) of the receiver's own quality. The map copies *which word changed* well (cosine 0.63 to 0.76) but the summary at the last position only weakly (0.24 to 0.31). The translator therefore passes its pre-set bar for Import and misses it for Export [P16].

As a positive control we swap one country for another in a template sentence. We translate the difference between the source model's states, add it to the receiver, and count a hit when the receiver's top answer becomes the swapped capital. The translated change works on **55%** of test cases for Export and **33%** for Import, against at most 0.4% for a random vector, a random map and another pair's difference (Figure 6.2) [P16].

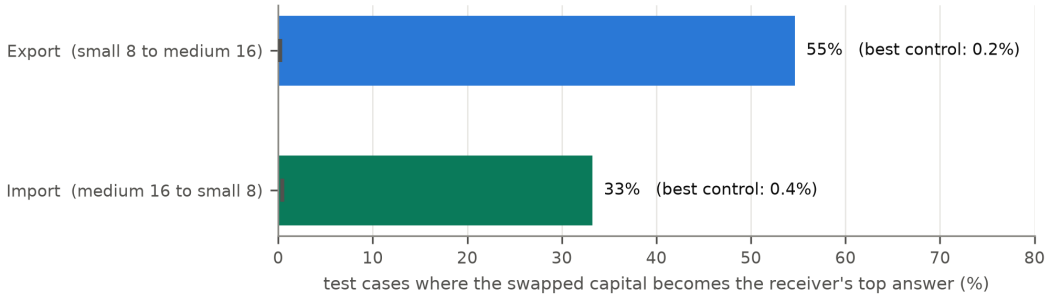


Figure 6.2: The positive control, a country swap [P16]. **How to read:** each bar is the share of test cases where the receiving model's top answer becomes the capital of the swapped country after the translated change is added. The small tick shows the best control (a random vector, a random map or another pair's difference). Both bars are far above their controls, so the channel carries information in both directions.

6.3 Export: a Real but Weak Effect

For each of the 88 primary test records we take the edit tuned in small, pass it through the ridge map and add it to medium at layer 16 (Table 6.1, Figures 6.3 and 6.4). The effect is real, graded and specific. The target's median rank in medium falls from 142 to **14** and it becomes the top answer on **8%** of records. Reworded prompts favour the new answer on 52% against 36% unchanged, and neighbours lose only 4 points. A plain word push makes the target first on 83% of records, but only 23% of neighbours keep the true answer, and stating the fact in medium's prompt costs 23 points of neighbours. The pre-set bar (top-1 of at least 15%, and 10 points above the strongest control) was not met [P16].

Table 6.1: Export of the edit through the ridge map (88 primary test records; dose chosen on dev) [P16].

Arm	Top-1	Median rank	Rewordings	Neighbours
Medium unchanged	0%	142	36%	67%
Translated edit	**8%**	**14**	**52%**	**63%**
Random vector	0%	157	38%	66%
Another record's edit	0%	128	36%	65%
Word push (uses the answer)	83%	1	82%	23%
Fact stated in medium's prompt	–	–	97%	44%

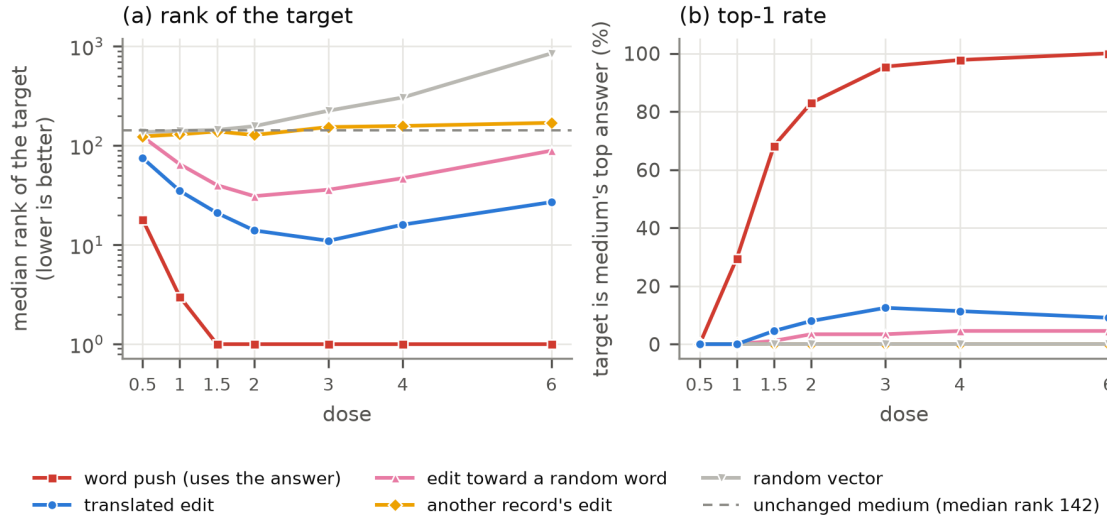


Figure 6.3: How the effect depends on the dose [P16]. **How to read:** the horizontal axis is the dose, the strength with which the translated change is added. (a) Median rank of the target (log scale, lower is better), with the dashed line showing unchanged medium. (b) Share of records where the target becomes medium's top answer. The translated edit (blue) improves clearly above the controls but stays far from the word push (red), which uses the answer itself.

6.4 Searching for a Better Translator

Since the effect was real but weak, we tried seven changes, each with its pass rule written first (Figure 6.5; Appendix B). Injecting at layers 4, 8, 12, 16 and 20 gave 0, 5, 5, 8 and 1%, and splitting the dose over several layers did no better, so the layer is not the limit. A neural map fits ordinary text much better (R^2 up by 0.09 to 0.11) but exports no better (8%), so a better fit to text does not help. A **map trained on edits** (method 3) roughly doubled the result to 18% top-1 (median rank 14 down to 7, paired Wilcoxon test [36], $p = 5 \times 10^{-9}$) and stayed specific to the record. More training edits, longer training and a direct rank-1 term in the loss all left it at about 20% [P16, P17].

Why it plateaus. With the rank-1 term, the map learned its 289 training targets (95% top-1 on them) but not unseen words (16%), which means it memorises. When we gave it 1,221 more edits toward 759 random target words, the memorisation went away but unseen records

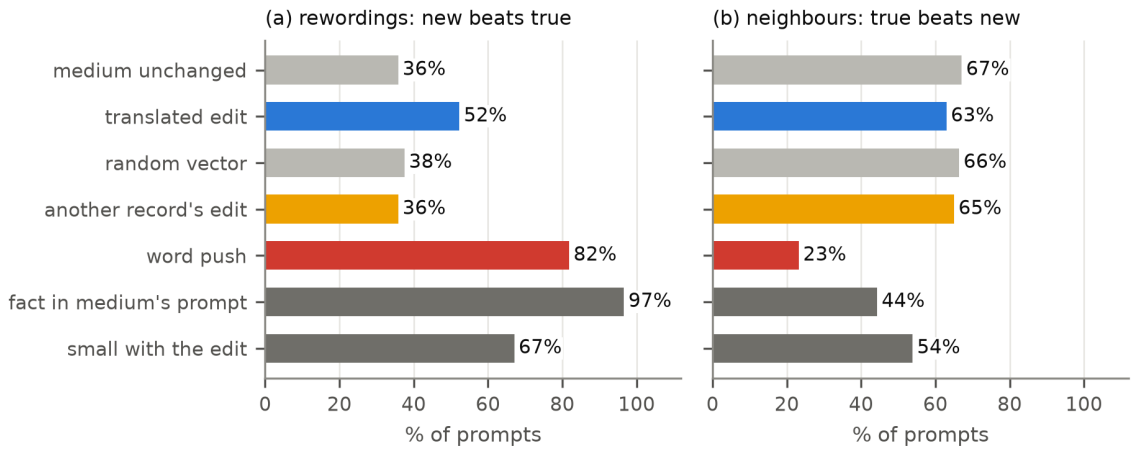


Figure 6.4: Effect on other prompts, Export through the ridge map [P16]. **How to read:** (a) share of reworded prompts where the new answer beats the true one (higher means the edit carries over). (b) Share of neighbour prompts where the true answer still wins (higher means less damage). The translated edit improves rewordings (52% against 36%) and spares neighbours (63% against 67%). The word push wins on rewordings but ruins neighbours (23%), and the fact in the prompt does so to a lesser degree.

got *worse*, from 20% to 14% (Figure 6.6). The loss asks medium to copy small's shift and not to reach rank 1, so a better copy does not help. **Per-sentence tuning is a ceiling.** Re-tuning the multipliers against medium's output reaches the top answer on 95% of records (84% on all 150), with the same neighbour cost as the source edit inside small (8 points against 9). So the channel can carry a rank-1 edit, and what fails is the compatibility of an edit tuned for small with medium. The ceiling's solution points only partly in the direction of the original edit (cosine 0.51 against 0.02 by chance), so a map that merely transforms the given edit cannot supply what is missing [P16, P17].

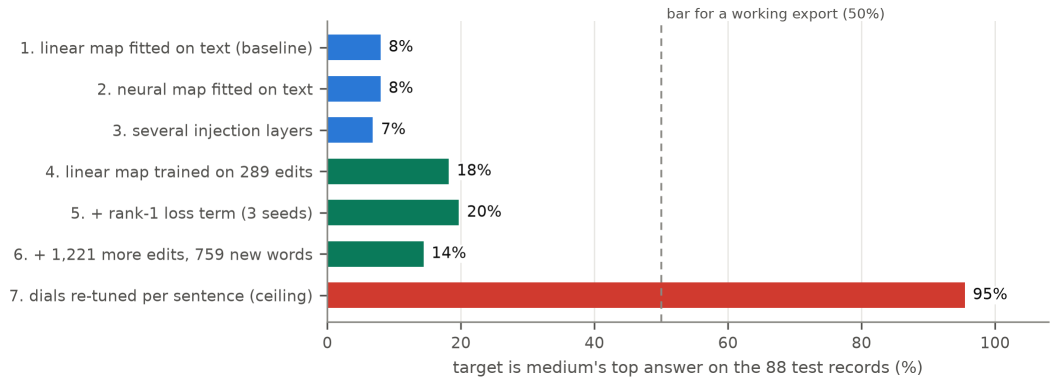


Figure 6.5: Top-1 of every export variant on the same 88 test records [P16, P17]. **How to read:** each bar is one version of the translator, from the first (left) to the last tried (right); taller is better. Blue: map fitted on text. Green: map trained on edits. Red: per-sentence tuning against medium's output (a ceiling, not a transfer). Every transfer variant stops at about 20%, far below the 50% pass bar.

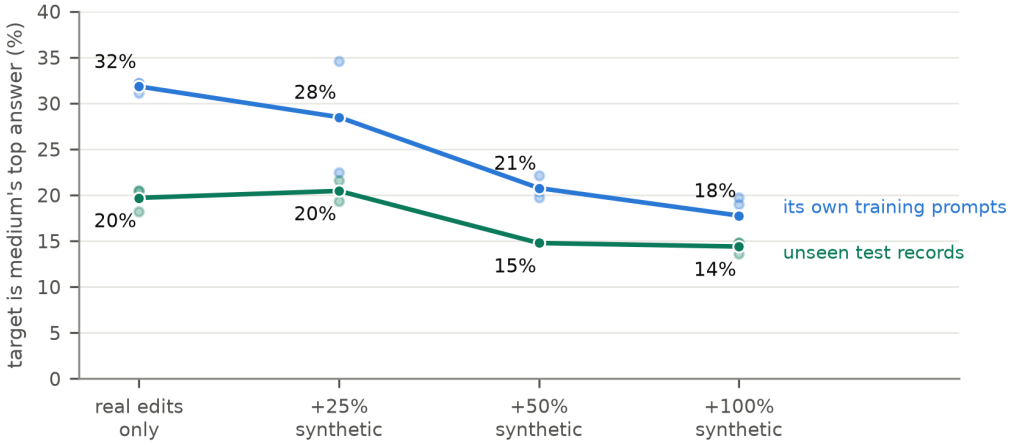


Figure 6.6: More and more varied training edits did not help [P17]. **How to read:** the horizontal axis is the amount of extra, synthetic training edits added to the 289 real ones. The blue line is the share of the map's own training prompts where the target becomes medium's top answer, the green line the same on unseen test records. Training performance falls from 32% to 18% as the memorisation disappears, but test performance does not rise: it falls from 20% to 14%.

6.5 What Is Carried: a Fact or a Push?

The question from Section 4.11 applies to Export too. The gain on reworded prompts has to be measured against each arm's *own* unchanged level, since a random word starts lower (25% against 36%). Measured this way, the per-sentence multipliers gain 24.4 points for the real target and 17.0 for a random word (difference +7.4, 95% bootstrap interval [53] 0.0 to +15.3, $p = 0.031$). The map trained on edits gains 19.9 and 18.2 ($p = 0.36$) (Figure 6.7). Neither difference passes the $p < 0.01$ line we fixed beforehand. What carries over to reworded prompts is a push toward the targeted word, not something specific to the fact [P17].

In one more control we repeated the per-sentence tuning with 200 random directions instead of the 200 SAE directions. The SAE directions reached 95% and the random ones 81% (Figure 6.8). So the SAE makes the tuning more effective, but it is not what makes steering possible. This agrees with benchmarks in which simple baselines steer as well as SAEs [12].

6.6 Import: Medium's State into Small

We use the 21,789 single-token CounterFact facts left after removing the development and pilot records. Medium alone gets 9.9% of them right and small gets 8.3%, and 780 facts are right in medium and wrong in small. **Full replacement** of small's layer-8 state by medium's translated state rescues 262 of those 780 (33.6%, with controls at most 0.8%). But it breaks 22% of the facts both models knew and 70% of those only small knew, so small's accuracy falls from 8.30% to 7.17%. The pre-set bar (50% rescue, accuracy half-way to medium, and 90% of known facts kept) was not met, although the controls show that real information does

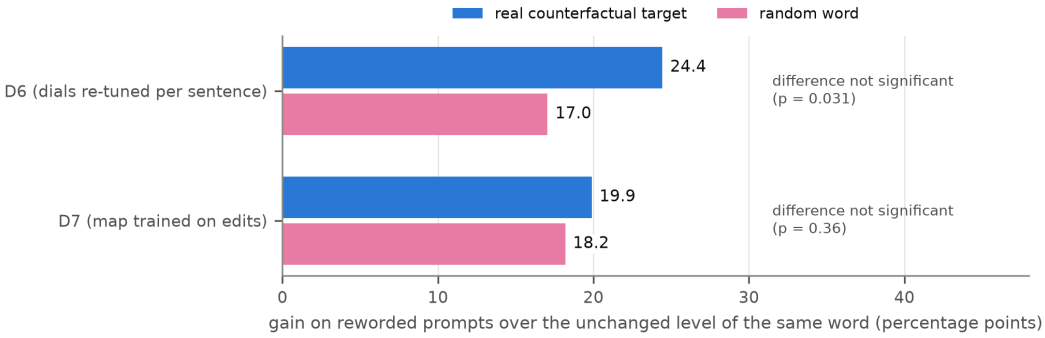


Figure 6.7: Is the gain on reworded prompts specific to the fact? [P17]. **How to read:** each bar is the gain in percentage points over the same word's unchanged level. Blue is the real counterfactual target, pink a random word. Two ways of making the edit are shown. In neither is the blue bar significantly longer than the pink one ($p = 0.031$ and $p = 0.36$ against a bar of $p < 0.01$), so the edit pushes toward whatever word it is aimed at.

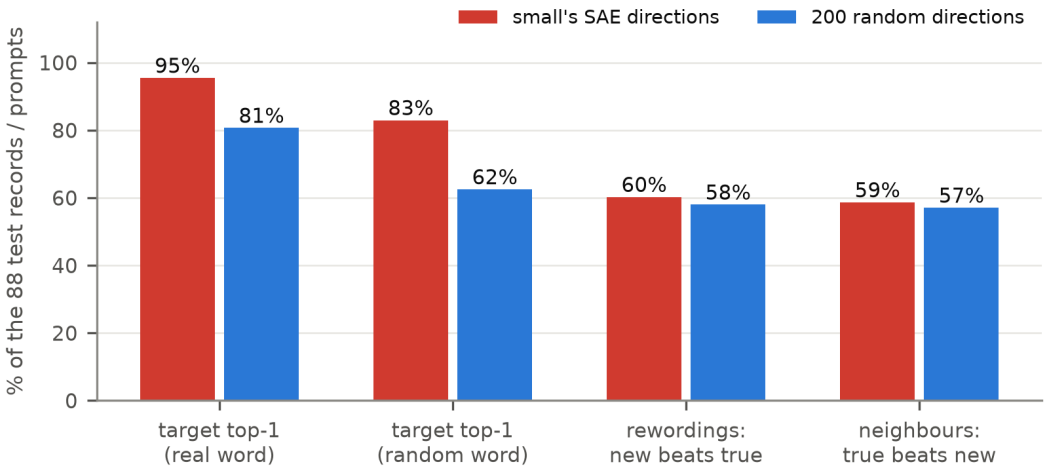


Figure 6.8: Do the SAE's directions matter? [P17]. **How to read:** red bars use the SAE's 200 decoder directions and blue bars 200 random directions, with the same tuning. Per-sentence tuning on the 88 primary test records. The SAE directions are better at making the target the top answer (95% against 81%, and 83% against 62% for a random-word target), but rewordings and neighbours are almost the same, so most of the steering power is not in the SAE.

cross [P17].

We then tried a **softer mix**, with its rule written before we built it. We chose $\gamma \in \{0.1, \ldots, 1\}$ on one half of the facts and report the other half (10,895 facts). Mixing in half of medium's state ($\gamma = 0.5$) raises small's accuracy from 8.5% to **9.5%**, where medium alone gets 10.2%. It gains 162 facts and loses 53 (exact McNemar test [35], $p = 2.5 \times 10^{-14}$), keeps 98% of the facts both models knew and rescues 28.6% of the facts small gets wrong (Figure 6.9). This meets the rule (a gain of at least 0.5 points, 90% of known facts kept, $p < 0.01$) and is our first net-positive Import result. It is only a one-point gain, it needs medium to run on the same sentence, and its controls were not rerun [P17].

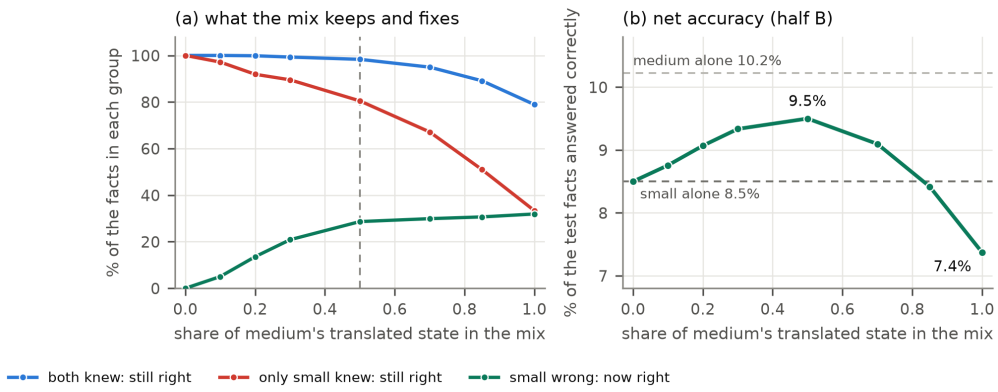


Figure 6.9: Import by mixing small's own state with medium's translated state [P17]. **How to read:** the horizontal axis is the share of medium's state in the mix (0 is small alone, 1 is full replacement). (a) What the mix keeps and fixes. (b) Net accuracy, which is best at a 50/50 mix and falls below small alone under full replacement.

6.7 What the Side Study Shows

A linear channel between GPT-2 Small and Medium exists and keeps the receiving model working. The change an edit makes in small moves medium's answers in a specific, graded way and without damaging its neighbours. But every translator that does not look at medium's output stops at about 20% top-1, far below the 50% bar, while looking at medium for every sentence reaches 95%. Together with the controls, this puts the limit on the *edit's compatibility* and not on the channel, the layer, the translator's capacity or the amount of data. What crosses is a push toward the targeted word and not a fact. The map follows earlier stitching work [40], and the limits of the study are listed in Chapter 8.

Chapter 7

Implementation

7.1 Software, Hardware and Application

The system is written in Python with PyTorch [20], TransformerLens 3.5.1 [13] (which gives every GPT-2 activation a named hook point and can run the model from a saved intermediate state), SAELens 6.45.3 [14] (which loads the `gpt2-small-res-jb` dictionaries [15]), Neuronpedia [18] for feature descriptions, and Streamlit [62] for the application. The ROME baseline uses the official code [60] with the Hugging Face `transformers` library [59]. The main-line studies ran on a GTX 1660 Ti PC (PyTorch 2.6.0, CUDA 12.4). Caching, the headroom study, the strength models and the transfer study ran on an RTX 4050 laptop (PyTorch 2.5.1, CUDA 12.1). The ROME baseline ran on a CPU [P15].

The application `experiment_app.py` has seven tabs and three tiers: the interface, the intervention engine, and the model and data layer (cached GPT-2, SAEs, fact sets and per-prompt caches) (Figure 7.1). The *Hybrid mute and boost* tab carries both editors behind a choice of editing method (Figure 7.2). You can pick the fixed sweep, with its strengths, cumulative sweep, pool refill, safety filter and GPU-batched switch, or gradient descent, with its steps, learning rate, margin and weights. It also has a live tracker, a stopwatch and a session-history record. The *Prototype lab* runs gradient descent with the optional additive edit, shows the baseline and edited text side by side, and applies the same edit to follow-up prompts (Figure 7.3). *Batch* runs studies in background workers, *Repair & SAE limit* shows the diagnostics of Section 4.7, *Sequential vs Batched Proof* times the two code paths, *Monosemanticity Analysis* inspects features, and *Session history* stores every run. The edit itself lives in `src/hooks.py` (Equation 3.2), `src/editing.py` (screening, causal ranking and the safety checks of Equation 3.4), `src/hybrid_runner.py` (the headless sweep, which reproduces the application exactly) and `src/gradient_editing.py` (Equations 3.5 to 3.9). The transfer study adds only new files (`src/transfer/`, and tools `00` to `27` in `tools/transfer/`), and the ROME baseline sits in `third_party/rome`, `src/rome_baseline` and `scripts/rome` [P15, P19]. We import older functions and

never edit them, so earlier results cannot change.

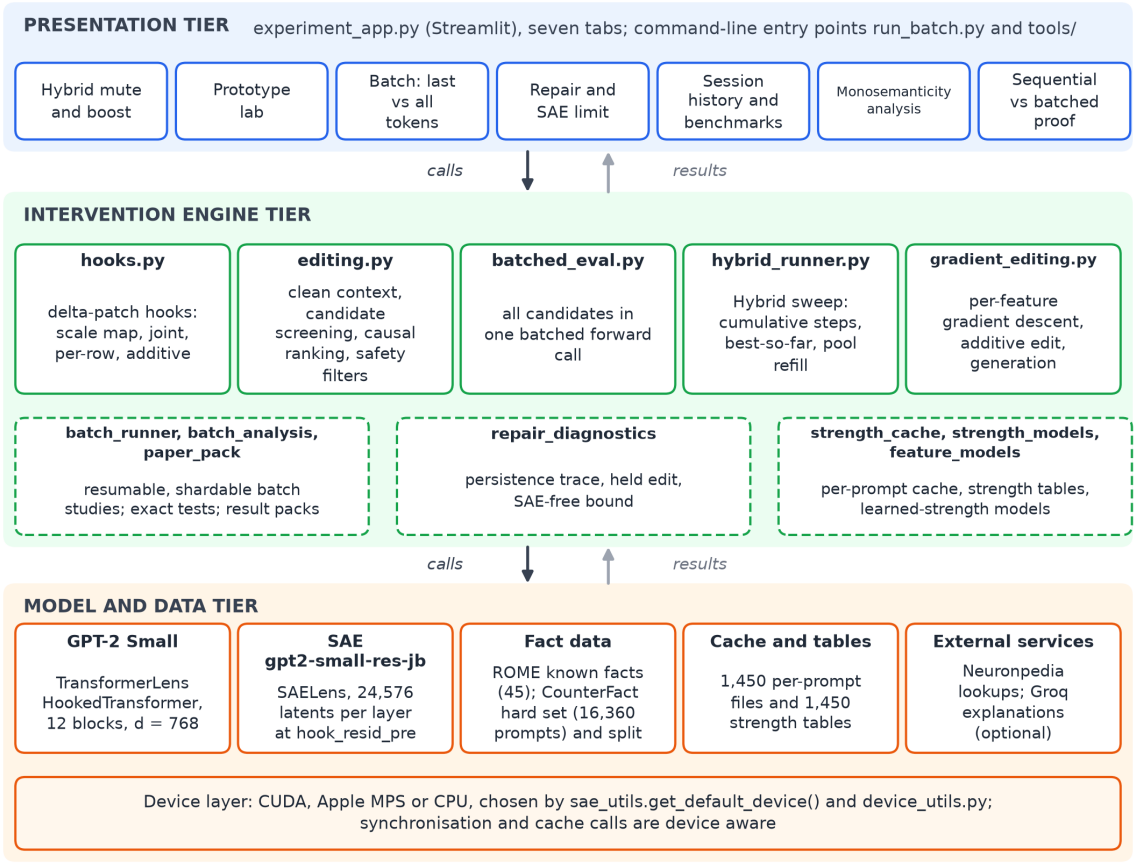


Figure 7.1: Software architecture [P19]. **How to read:** the three coloured bands are layers of the program. The top band is what the user sees (the seven tabs), the middle band holds the code that makes the edit and the studies, and the bottom band holds the models, data and caches. Arrows show that the upper layers call the lower ones and receive results.

7.2 Correctness Checks and Engineering Lessons

We checked every fast path against a slower reference before using it, and we kept the checks as scripts. Batched ranking matched sequential ranking (identical feature lists, probabilities to 10^{-9}, and 60 of 60 safety decisions). The cached-state path matched the full model (identical logits on one machine, within 10^{-5} on the other). A random per-feature edit through the cache path gave the same target rank as the real hook, and the gradient-descent tuning path agreed with the real hook in every recorded run. Built-in sanity checks stop a run if something does not match, and one of them caught an error in the cache builder (it passed a multiplier of 1.0 where 0.0 was meant) before any data was written [P5, P10, P13]. Three problems could have ruined results without raising any error.

- **A thread race removed the start-of-text token.** In TransformerLens, tokenising with

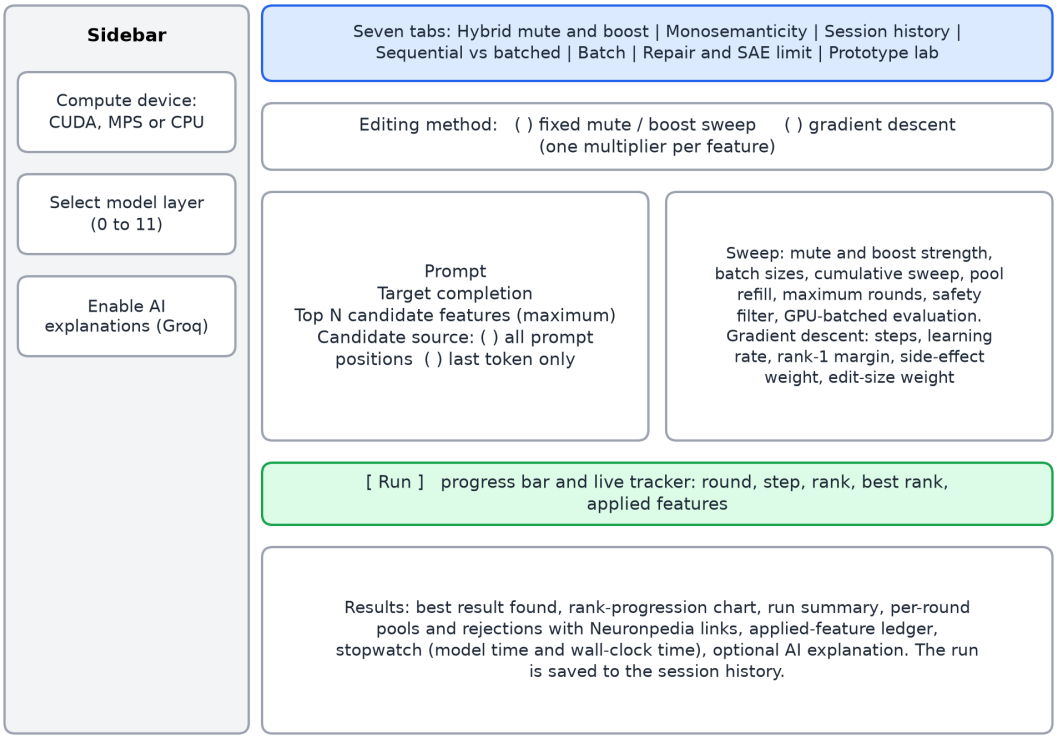


Figure 7.2: Layout of the Hybrid mute and boost tab [P19]. **How to read:** the left panel selects the device and layer, and the centre holds the prompt, the choice of editing method and its settings. The Run button starts the edit and the lower panels show the result, the rank progression and the applied features.

the start token switched off temporarily changes a setting shared by the whole model. When two Streamlit sessions overlapped, the wrong value could be written back for good, and every later prompt was tokenised without its start token (rank 28 instead of 22 on one prompt). Of 57 recorded application runs, 16 had wrong baselines, all from one server process, and none of them is used in a table. The fix tokenises directly and never touches the model's configuration [P14].

- **A timer measured the network** (46 s against about 10 s of model time), and **chained hooks disagreed with the combined check** (rank 66 against 59). We moved the look-ups out of the timed region and now use one combined hook [P6].
- **GPU memory.** A padded training set of 7,550 texts (3.8 GB) nearly filled the 6 GB card and made runs five to ten times slower with no error at all (6.2 hours instead of one). Training tensors now stay on the CPU [P17].

7.3 Reproducibility and Development Process

Every tool prints its pass rule, saves its raw output as a log, and writes a result file with the per-run details. The logs and results of every experiment are copied into a data pack under version

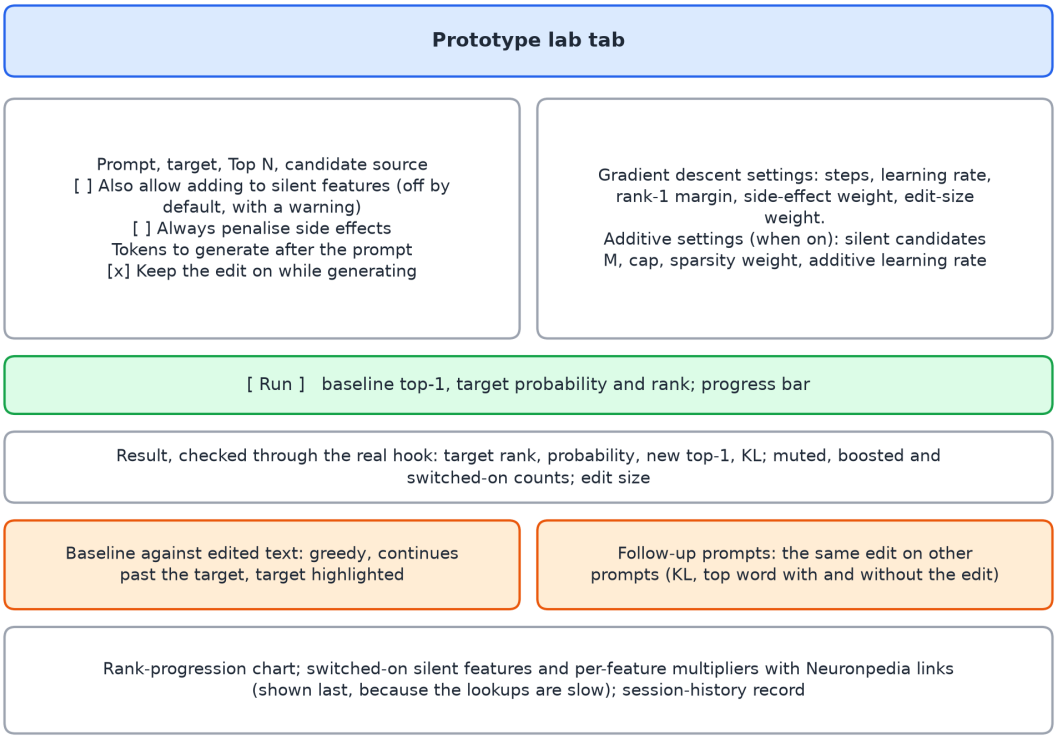


Figure 7.3: Layout of the Prototype lab tab [P14]. **How to read:** the upper boxes are the inputs and settings, the green box runs the edit, and the lower boxes show the checked result, the baseline and edited text side by side, and the follow-up prompts that receive the same edit.

control, and the figures in this report are drawn from that pack by scripts that run no model (Appendix D). We optimised in one bounded step at a time: find one bottleneck, measure the reference, make one change, check equivalence on a fixed case, log it, and only then approve the next step [P5, P19].

The software and this report were developed with AI coding assistants (Antigravity AI earlier, Claude later) under the authors' direction. The authors chose the questions, pass bars, controls and each next experiment, and ran or approved every long job. The assistants wrote and tested code, drafted documentation, and were asked to record failures as carefully as successes. [TODO: check this statement against the college's policy; delete it if disclosure is not required]

Chapter 8

Discussion, Conclusion and Future Work

8.1 What the Work Shows

The research questions of Chapter 1 have the following answers, each backed by the project record named in brackets.

RQ1 and RQ2: what works and what limits it. The sweep was held back by the supply of features, not by the selection rule. Taking candidates from all prompt positions made the pool about five times larger and raised the prompts at rank 1 from 4 to 10 of 13 [P7]. Choosing the right features and strengths mattered more than stronger interventions. Muting the competitor's drivers and adding boosts gave the early gains [P3]. We saw no sign that later layers undo the edit, and a free residual edit reached rank 1 where the SAE edit did not, so part of the limit lies in the SAE basis [P9].

RQ3: choosing the edit. One better global strength pair lifted a stand-in success rate from 25% to 39%, and choosing strengths per prompt added nothing we could measure [P12]. Tuning one multiplier per feature on the prompt itself reached 271 of 300 on the real model, against 161 and 94 for the sweep, with lower same-prompt side effects and 5 to 6.5 times less compute [P13].

RQ4: what rank 1 means. Rank 1 turned out to be a weak test. The additive edit forced random words to rank 1 on 20 of 20 prompts, and an optimiser with enough freedom can reach any output you pick [P14, 46, 47, 48]. Being able to steer a target to the top says nothing about what the model knows, in either direction. The edit itself is a push toward the answer's output direction. It carries little beyond the prompt it was tuned on, and it mostly ignores an instruction not to say the word [P14].

RQ5: the outside check. ROME runs on GPT-2 Small with the official code, and its causal trace puts the strongest effect at the last token in the upper layers (8 to 11), where our edit layer 8 sits at the start of that range [P15]. Scored with ROME's own metrics, our edit raises the reworded-prompt score from 40% to 67% and costs 9.5 points of neighbour score, while ROME's layer-3 edit raises it from 33% to 98.5% and costs 5 points, on different records

(Figure 5.7). This agrees with the push finding. It is not a controlled comparison.

RQ6: the application. A linear channel between GPT-2 Small and Medium keeps the receiving model working. An exported edit moves the target in a specific way but rarely all the way to rank 1, what crosses is a push and not a fact, and mixing in half of Medium's state raises Small's accuracy by one point [P16, P17].

Four habits paid off: controls first, fair comparisons, pass bars fixed before each run, and corrections made in the open. Withdrawn claims are marked in the journal where they happened [P14, P18].

8.2 Threats to Validity and What Is Not Shown

- We use one model family, one older SAE release, and layer 8 in all later studies. The early studies use small, templated prompt sets (13 to 131 prompts), and like CounterFact they treat prompts as independent, without correction for multiple comparisons.
- Success means rank 1 on the next token, and the median target probability at rank 1 is only 7.7% for gradient descent. We measured side effects on the same prompt, on 20 unrelated prompts and in 40-prompt tests, with one seed, no confidence intervals, and true targets only in the generation and instruction tests [P13, P14].
- The headroom and the learned strengths are stand-ins measured on cached states, and we never ran the real sweep with the learned strengths. The side study has 88 primary test records.
- **We have not compared the approach with ROME, MEMIT, in-context editing, prompting or difference-of-means steering on one record list**, so nothing here says how it ranks against them. The partial cross-check of Section 5.7 uses different record sets and different ways of re-applying each edit.
- The ROME baseline was run on 100 development records with one seed, and the choice of layer 3 rests on differences within noise (Section 5.5). The causal trace covers 97 facts that GPT-2 Small already knows well.
- Some sources were read through their abstracts only (Chapter 2), so descriptions of those works are limited to what the abstracts state.

8.3 Conclusion

FeatureScalpel set out to repair suppressed factual recall in GPT-2 Small through the features of a sparse autoencoder, without changing a single weight. After many attempts that mostly failed in useful ways, we arrived at a transient edit that brings the true answer to rank 1 on

Table 8.1: Claims for the research paper and the evidence behind each.

Claim	**May be claimed?**	**Evidence**
Gradient descent brings the true answer to rank 1 on 271 of 300 held-out prompts, against 161 and 94 for the sweep	Yes	Equal-budget comparison with exact paired tests [P13]
It does so with lower same-prompt side effects and less compute	Yes, with the caveat that the loss contains a KL term	[P13]
The edit recovers or repairs the model's knowledge	No	Additive control: 20 of 20 random words at rank 1 [P14]
The edit is specific to the fact it targets	No	Rewordings gain equal for real and random targets [P14, P17]
ROME runs on GPT-2 Small with the official code and its trace points to the upper layers at the last token	Yes	[P15]
Our edit scored with ROME's PS and NS is weaker than ROME's on rewordings	As an indication only	Different record sets; Section 5.7
Our approach beats or matches ROME, MEMIT or in-context editing	No	Not compared; Section 5.8
A change made in one model can be carried to another as a push	Yes, with the controls	Export ladder and push test [P16, P17]
Mixing in half of Medium's state raises Small's accuracy by one point	Yes, as a small effect	Held-out half of 21,789 facts; controls not rerun [P17]

271 of 300 held-out prompts, quickly and with small side effects on the same prompt [P13]. An outside check against ROME's official implementation and metrics suggests that the edit generalises less to reworded prompts than a weight edit does. The applied side study found that the change an edit makes can be carried between models as a push, that importing a larger model's state gives a small gain, and that a fact does not cross. The results support the claim that a transient SAE-feature edit can raise a suppressed answer to the top, and that tuning a multiplier per feature does so more often, faster and with lower same-prompt side effects than a hand-built sweep. They do not support the claim that such edits recover knowledge, that they are specific to a fact, or that they beat weight editing, in-context editing or prompting.

8.4 Decisions for the Research Paper

The research paper will be built from this report. Table 8.1 separates what it may claim from what it may not, with the evidence for each, so that nothing is written that the record does not support.

The decisions that remain open are these. (1) Whether to run the head-to-head comparison with ROME before submission. The paper's claims are safe without it, but a reviewer will ask.

(2) Whether to repeat the main comparison on more than one seed and report intervals. (3) Whether the layer choice should be re-examined on a larger set, since the plateau at layers 6 to 9 is not a proven optimum. (4) Whether to present the transfer study in the paper or to keep it as a separate short report.

8.5 Future Work

1. **A head-to-head comparison** with ROME, MEMIT, in-context editing and difference-of-means steering, following the plan of Section 5.8: one frozen record list, one scoring script, existing implementations [30, 60], and all timings on one machine.
2. **A specificity benchmark.** For every method, on the same prompts, it should measure whether the target reaches rank 1, carries over to rewordings, leaves unrelated prompts and generated text alone, and respects an instruction, with random words of matched starting rank as controls.
3. **More strengths, transfer and scale.** We would run the learned strengths in the real sweep, rerun the controls for the Import mix, and try other model pairs that have pretrained SAEs on both sides, and larger models with pretrained SAEs [17].

References

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.

[2] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 17359–17372.

[3] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[4] A. Gupta, A. Rao, and G. Anumanchipalli, "Model editing at scale leads to gradual and catastrophic forgetting," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024; arXiv:2401.07453.

[5] N. Elhage, T. Hume, C. Olsson, *et al.*, "Toy models of superposition," *Transformer Circuits Thread*, 2022.

[6] T. Bricken, A. Templeton, J. Batson, *et al.*, "Towards monosemanticity: Decomposing language models with dictionary learning," *Transformer Circuits Thread*, 2023.

[7] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse autoencoders find highly interpretable features in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[8] A. Templeton, T. Conerly, *et al.*, "Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet," *Transformer Circuits Thread*, 2024.

[9] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid, "Steering language models with activation engineering," *arXiv preprint arXiv:2308.10248*, 2023.

[10] N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner, "Steering Llama 2 via contrastive activation addition," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 15504–15522.

[11] A. Zou, L. Phan, S. Chen, *et al.*, "Representation engineering: A top-down approach to AI transparency," *arXiv preprint arXiv:2310.01405*, 2023.

[12] Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts, "AxBench: Steering LLMs? Even simple baselines outperform sparse autoencoders," in *Proc. International Conference on Machine Learning (ICML)*, 2025; arXiv:2501.17148.

[13] N. Nanda and J. Bloom, "TransformerLens," GitHub repository, 2022. [Online]. Available: https://github.com/TransformerLensOrg/TransformerLens

[14] J. Bloom, C. Tigges, A. Duong, and D. Chanin, "SAELens," GitHub repository, 2024. [Online]. Available: https://github.com/decoderesearch/SAELens

[15] J. Bloom, "Open source sparse autoencoders for all residual stream layers of GPT2 small," *AI Alignment Forum*, 2024.

[16] J. Kramár, T. Lieberum, R. Shah, and N. Nanda, "AtP*: An efficient and scalable method for localizing LLM behaviour to components," *arXiv preprint arXiv:2403.00745*, 2024.

[17] T. Lieberum, S. Rajamanoharan, A. Conmy, *et al.*, "Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2," *arXiv preprint arXiv:2408.05147*, 2024.

[18] J. Lin and J. Bloom, "Neuronpedia: Interactive reference and tooling for analyzing neural networks," 2023. [Online]. Available: https://www.neuronpedia.org

[19] S. Heimersheim and N. Nanda, "How to use and interpret activation patching," *arXiv preprint arXiv:2404.15255*, 2024.

[20] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[21] D. Arad, A. Mueller, and Y. Belinkov, "SAEs are good for steering – if you select the right features," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025; arXiv:2505.20063.

[22] L. Smith, S. Rajamanoharan, A. Conmy, C. McDougall, T. Lieberum, J. Kramár, R. Shah, and N. Nanda, "Negative results for SAEs on downstream tasks and deprioritising SAE research (GDM mech interp team progress update #2)," *LessWrong*, Mar. 2025. [Online]. Available: https://www.lesswrong.com/posts/4uXCAJNuPKtKBsi28/negative-results-for-saes-on-downstream-tasks

[23] M. Chaudhary and A. Geiger, "Evaluating open-source sparse autoencoders on disentangling factual knowledge in GPT-2 small," *arXiv preprint arXiv:2409.04478*, 2024.

[24] M. Cui, L. Shen, and X. Yang, "SAE interventions are unreliable: Post-intervention recovery of suppressed behavior," *arXiv preprint arXiv:2606.18322*, 2026.

[25] E. Farrell, Y.-T. Lau, and A. Conmy, "Applying sparse autoencoders to unlearn knowledge in language models," *arXiv preprint arXiv:2410.19278*, 2024.

[26] S. Chalnev, M. Siu, and A. Conmy, "Improving steering vectors by targeting sparse autoencoder features," *arXiv preprint arXiv:2411.02193*, 2024.

[27] Y. Zhao, A. Devoto, G. Hong, X. Du, A. P. Gema, H. Wang, X. He, K.-F. Wong, and P. Minervini, "Steering knowledge selection behaviours in LLMs via SAE-based representation engineering," in *Proc. NAACL*, 2025; arXiv:2410.15999.

[28] P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun, "Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[29] C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang, "Can we edit factual knowledge by in-context learning?" in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023; arXiv:2305.12740.

[30] P. Wang, N. Zhang, B. Tian, *et al.*, "EasyEdit: An easy-to-use knowledge editing framework for large language models," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024; arXiv:2308.07269.

[31] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller, "Sparse feature circuits: Discovering and editing interpretable causal graphs in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2025; arXiv:2403.19647.

[32] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[33] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[34] nostalgebraist, "Interpreting GPT: the logit lens," *LessWrong*, Aug. 2020. [Online]. Available: https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens

[35] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[36] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[37] K. Lenc and A. Vedaldi, "Understanding image representations by measuring their equivariance and equivalence," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[38] Y. Bansal, P. Nakkiran, and B. Barak, "Revisiting model stitching to compare neural representations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[39] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proc. International Conference on Machine Learning (ICML)*, 2019.

[40] A. Chen, J. Merullo, A. Stolfo, and E. Pavlick, "Transferring linear features across language models with model stitching," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025; arXiv:2506.06609.

[41] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[42] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[43] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[44] N. Subramani, N. Suresh, and M. E. Peters, "Extracting latent steering vectors from pretrained language models," in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022, pp. 566–581.

[45] J. Dunefsky and A. Cohan, "One-shot optimized steering vectors mediate safety-relevant behaviors in LLMs," in *Conference on Language Modeling (COLM)*, 2025; arXiv:2502.18862.

[46] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2014.

[47] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[48] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[49] N. Elhage, N. Nanda, C. Olsson, *et al.*, "A mathematical framework for transformer circuits," *Transformer Circuits Thread*, 2021.

[50] S. Kullback and R. A. Leibler, "On information and sufficiency," *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.

[51] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[52] W. J. Dixon and A. M. Mood, "The statistical sign test," *Journal of the American Statistical Association*, vol. 41, no. 236, pp. 557–566, 1946.

[53] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[54] J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber, "Investigating gender bias in language models using causal mediation analysis," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[55] J. Pearl, "Direct and indirect effects," in *Proc. 17th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2001, pp. 411–420.

[56] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 5484–5495.

[57] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[58] J. Li, M. Galley, C. Brockett, J. Gao, and B. Dolan, "A diversity-promoting objective function for neural conversation models," in *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2016, pp. 110–119.

[59] T. Wolf, L. Debut, V. Sanh, *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

[60] K. Meng *et al.*, "ROME: Rank-One Model Editing, official implementation," GitHub repository, commit 0874014 (MIT licence). [Online]. Available: https://github.com/kmeng01/rome

[61] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT: project page, CounterFact and known-facts data." [Online]. Available: https://rome.baulab.info

[62] Streamlit Inc., "Streamlit: A faster way to build and share data apps," software. [Online]. Available: https://streamlit.io

Project records. Results that come from our own experiments are cited to the records below (labels P1 to P19). All are files in the project repository (branch `cross-model-transfer`); every figure and table in this report is drawn from them (Appendix D).

[P1] FeatureScalpel project, "Research Journal, entry 1: project progress and roadmap (first prototype, causal selector, eligibility rule)," July 2026. `docs/Research_Journal/1.md`.

[P2] FeatureScalpel project, "Research Journal, entries 2 and 3: iterative ablation of the target's helpers and of the competitor's drivers," 19–20 July 2026. `docs/Research_Journal/2.md, 3.md`.

[P3] FeatureScalpel project, "Research Journal, entries 4 to 8: whole-combination safety, weighted multi-competitor muting, mute-and-boost diagnostics and parameter sweep," 24–25 July 2026. `docs/Research_Journal/4.md` to `8.md`.

[P4] FeatureScalpel project, "Research Journal, entries 9 to 11: SAE dictionary audit, layer-depth benchmark, benchmark infrastructure and stage profiling," 4–6 August 2026. `docs/Research_Journal/9.md` to `11.md`; data `benchmark_results/layer_benchmark_*.json`, `benchmark_results/layer_benchmark_20260909_summary.csv`.

[P5] FeatureScalpel project, "Research Journal, entries 12 to 15: delta patching, hardware comparison, execution-cost audit, GPU batching of candidate evaluation," 9 August to 18 September 2026. `docs/Research_Journal/12.md` to `15.md`.

[P6] FeatureScalpel project, "Research Journal, entries 16 to 19: best-so-far tracking, cumulative sweep, combined hook, pool refill," 18–24 September 2026. `docs/Research_Journal/16.md` to `19.md`.

[P7] FeatureScalpel project, "Research Journal, entry 20: candidate source, last token against all prompt positions," 25–26 September 2026. `docs/Research_Journal/20.md`; data `benchmark_results/candidate_source_study/`.

[P8] FeatureScalpel project, "Safety-filter study (journal entry 21 placeholder and result packs)," 26 September 2026. `docs/Research_Journal/packs/entry21_*`; specifications `data/safety_filter_spec_*.json`.

[P9] FeatureScalpel project, "Research Journal, entry 22: do later layers undo the edit, is the SAE the limit," 30 September 2026. `docs/Research_Journal/22.md`; data `docs/Research_Journal/packs/repair_diagnostics/`.

[P10] FeatureScalpel project, "Research Journal, entries 23 and 24: CounterFact hard set, split, per-prompt cache and headroom measurement," 1–2 October 2026. `docs/Research_Journal/23.md, 24.md`; data `data/counterfact_*.json`, `docs/Research_Journal/packs/headroom/`.

[P11] FeatureScalpel project, "Research Journal, entry 25: literature check and assessment of the research direction," 1 October 2026. `docs/Research_Journal/25.md`.

[P12] FeatureScalpel project, "Research Journal, entries 26 and 27: learned strengths (PromptNet, FeatureNet)," 2–3 October 2026. `docs/Research_Journal/26.md, 27.md`; data `docs/Research_Journal/packs/strength_models/`.

[P13] FeatureScalpel project, "Research Journal, entries 28 and 29: gradient-descent editing and the equal-budget comparison with the sweep," 3 October 2026. `docs/Research_Journal/28.md, 29.md`; data `docs/Research_Journal/packs/gradient_descent/`, `packs/sweep_vs_gradient/`, `images/e29_figure_data.json`.

[P14] FeatureScalpel project, "Research Journal, entry 30: additive edits, random-target control, reworded-prompt, generation and instruction tests, Prototype lab," 4 October 2026. `docs/Research_Journal/30.md`; data `docs/Research_Journal/packs/additive_control/`, `packs/generalisation/`, `packs/generation_quality/`.

[P15] FeatureScalpel project (branch `rome-factual-editing`), "ROME baseline: progress log, causal-tracing summary and edit-layer sweep on GPT-2 Small," 7 October 2026. `docs/rome_baseline/PROGRESS.md`, `data/comparison/trace_summary.json`, `data/comparison/gpt2_knowns.json`, `benchmark_results/rome_vs_scalpel/layer_sweep_dev.csv`; code `third_party/rome/`, `src/rome_baseline/`, `scripts/rome/`.

[P16] FeatureScalpel project, "Research Journal, entries 31 to 33: translator between GPT-2 Small and Medium, country swap, controlled Export, search for a better translator," 9–10 October 2026. `docs/Research_Journal/31.md` to `33.md`; data `docs/Research_Journal/packs/cross_model_transfer/`.

[P17] FeatureScalpel project, "Research Journal, entries 34 to 37: Import, varied-edit training, softer Import, SAE-basis control," 10–11 October 2026. `docs/Research_Journal/34.md` to `37.md`; data `docs/Research_Journal/packs/cross_model_transfer/`.

[P18] FeatureScalpel project, "Cross-model transfer plan with pass bars (gates) written before each run," October 2026. `docs/cross_model_transfer/PLAN.md`.

[P19] FeatureScalpel project, "Research roadmap, handoff notes and architecture notes." `docs/research_roadmap.md`, `docs/handoff_*.md`, `docs/architecture.md`.

Appendices

Appendix A: Research Journal and Report Sections

The journal (`docs/Research_Journal/`, entries 1 to 37) records every step with its tables, rules, mistakes and corrections. Result files and raw logs of the later entries are in `docs/Research_Journal/packs/`.

Table A.1: Journal entries and where they appear in this report.

Entries	Content	Section
1	First prototype, causal selector, eligibility rule, roadmap	4.1
2, 3	Target-helper and competitor ablation (1 and 3 of 22)	4.2
4, 6, 7, 8	Weighted multi-competitor muting, aggregation, decay profile, top-K	4.3
5	Hybrid mute and boost, rank regression	4.4
9, 10, 11	SAE audit, layer benchmark, caching and profiling	4.5
12, 13, 14, 15	GPU, redundant-pass removal, batching (186 to 6 calls), batched application	4.5
16, 17, 18, 19	Best-so-far, cumulative sweep, combined hook, pool refill, all-position candidates	4.6
20, 21	Candidate-source study (10 of 13 against 4 of 13); safety-filter study	4.6
22	Repair and SAE-limit diagnostics	4.7
23, 24	Learned-strength plan; CounterFact hard set, split, cache, headroom	4.8
25	Literature check	4.7, Ch. 2
26, 27	Learned strengths (PromptNet, FeatureNet)	4.9
28, 29	Gradient-descent editor; equal-budget comparison (271, 161, 94 of 300)	4.10
30	Additive edit, random-word control, rewording, generation and instruction tests, start-token bug	4.11, 4.12
ROME log	ROME baseline on GPT-2 Small (branch `rome-factual-editing`, 7 October): causal tracing, layer sweep, metrics	Ch. 5
31 to 37	Cross-model transfer: translators, country swap, Export, search ladder, Import, SAE control	Ch. 6

Appendix B: Pass Bars of the Side Study and Their Verdicts

Every bar was written into the project plan, and committed, before the run that judged it; none was moved afterwards. "Edit's own" means the effect that the source edit has inside small

itself. Verdicts are as printed by the tools.

Table B.1: Pass bars ("gates") and their verdicts.

Step	Bar fixed before the run	Result	Verdict
Gate 1 (A2)	Translator: difference cosine at the last position at least 0.3	Import pairs 0.31; Export pair 0.28	Import only
Gate 2 (B1)	Country swap beats the strongest control by a margin, paired test	55% and 33% against at most 0.4%	pass
Gate 3 (D)	Export top-1 at least 15% and 10 points above the strongest control	8%; margin 3 points	fail
Gate 4 (D5)	Best layer set at least 10 points above the best single layer, $p < 0.01$, top-1 at least 15%	7% against 8%; $p = 0.016$	fail
Gate 5 (D6)	Top-1 $\geq$ 50%; rewordings lift $\geq$ 80% of the edit's own; neighbours fall $\leq$ 5 points	95%; 0.91; 8 points	fail (neighbours)
Gate 6 (D7)	Top-1 $\geq$ 50%; rewordings lift $\geq$ 0.8; neighbours fall no more than the edit's own	18%; 0.74; 7 points	fail
Gate 7 (D10)	As Gate 6, plus real target 10 points above the random word	20%; 0.74; met; met	fail
Gate 8 (Import)	Rescue $\geq$ 50% of the 780 facts; accuracy halfway to medium; 10 points above controls ($p < 0.01$); 90% of known facts kept	33.6%; 7.2% against 9.1%; met; 78.3%	fail
Gate 9 (D13)	Gate 7 unchanged, on the full varied training set	14%; 0.62; met; met	fail
Rule E5	Soft Import: gain $\geq$ 0.5 points on held-out facts, 90% kept, $p < 0.01$	+1.0 point; 98.3%; $p = 2.5 \times 10^{-14}$	met
Rule S1	SAE basis "matters" if top-1 is higher by 10 points or rewordings lift by 5	+15 points; +2 points	matters
Rules S2, S3	Real target's rewordings lift larger than a random word's, $p < 0.01$	$p = 0.031$ (D6); $p = 0.36$ (D7)	not shown

Two notes written after the fact do not change any verdict: the 5-point neighbour line of Gate 5 was stricter than the source edit's own loss of 9 points inside small, so Gates 6, 7 and 9 use the edit's own loss; and the dose rule (dev top-1 on 28 records) was replaced by the dev log-rank gain from step D10 on, after step D9b showed it to be fragile.

Appendix C: Reproducing the Results

From the repository root, with `HF_HUB_OFFLINE=1` once the models are cached. The main line is on the branches `layer-intervention-benchmark`, `pool-refill-implementation` and `gradient-descent-editing`; the side study is on `cross-model-transfer`.

```
streamlit run experiment_app.py
python run_batch.py data/candidate_source_batch_spec.json
python tools/build_counterfact_set.py
python tools/split_counterfact_set.py
python tools/build_strength_cache.py
```

```
python tools/measure_headroom.py
python tools/train_strength_models.py
python tools/compare_sweep_vs_gradient.py --n-per-band 0
python tools/control_random_targets.py
python tools/transfer/00_setup_check.py   (then 01 to 27)
python scripts/rome/06_sweep_layers.py    (ROME environment, branch rome-
    factual-editing)
python tools/make_report_figures.py
python tools/make_report_v2_figures.py
python tools/make_report_v3_history_figures.py
python tools/make_report_v4_figures.py
```

In order: the application; the candidate-source study; the CounterFact hard set and split; the per-prompt cache and the headroom measurement; the strength networks; the 271, 161 and 94 of 300 comparison; the random-word control; the side study; the ROME layer sweep; and the four figure scripts (Appendix D).

Journal entries 31 to 37 give each side-study tool with its run time, and `docs/cross_model_transfer/PLAN.md` holds all pass bars and readings.

Appendix D: Where Every Figure Comes From

Every figure in the report is drawn by a script in `tools/` from stored data, and no figure script runs a model. Diagrams contain no data. Where a figure shows numbers that were printed in a journal entry and never stored as a result file, the table says so. The data labels P1 to P19 refer to the project records in the reference list. Paths are relative to `docs/Research_Journal/` unless they start with `data/` or `benchmark_results/`.

Table D.1: Figures, their data and the script that draws them.

Figure file	Data	Script
`h_stages`, `h_early`, `h_speed`	Numbers printed in journal entries 1 to 15 (P1 to P5), typed into the script header	`make_report_v3_history_figures`
`h_gd_flow`, `h_additive`, `v4_rome_compare`, `v4_rome_in_project`, `r_overview`	Diagrams, no data	`make_report_v3_history_figures`, `make_report_v4_figures`, `make_report_v2_figures`
`v4_algo`, `v4_arch`, `v4_ui_hybrid`, `v4_ui_proto`	Diagrams, no data	`make_report_figures` (first report)
`v4_elig`, `v4_pipeline`	Counts from P1 and from `data/counterfact_hard_set_summary.json` and `data/counterfact_split_summary.json` (P10)	`make_report_figures`
`v4_hybrid_sophia`, `v4_cand_pools`, `fig_candsrc`	`benchmark_results/candidate_source_study/` (P7)	`make_report_v4_figures`, `make_report_figures`
`v4_layer12`	`benchmark_results/layer_benchmark_20260909_summary.csv` (P4)	`make_report_figures`
`v4_filter`	`packs/entry21_*/tables/arms_*.csv` (P8)	`make_report_figures`
`v4_repair`	`packs/repair_diagnostics/merged_all_layers.json` (P9)	`make_report_v4_figures`

Figure file	Data	Script
fig_headroom	packs/headroom/headroom_20261002_144917.json (P10)	make_report_figures
fig_learned	packs/strength_models/results.json (P12)	make_report_figures
v4_gd_trajectories	packs/gradient_descent/session_20261003_015115.json (P13)	make_report_v4_figures
h_gd_bars, v4_gd_tradeoff	images/e29_figure_data.json (P13)	make_report_v3_history_figures, make_report_v4_figures
v4_gd_side	packs/sweep_vs_gradient/runs.jsonl (P13)	make_report_v4_figures
fig_additive, v4_reworded, v4_genquality	packs/additive_control/, packs/generalisation/, packs/generation_quality/ (P14)	make_report_figures
v4_rome_eiffel	Original plot of the ROME branch; the underlying trace was not stored (P15)	copied unchanged
v4_rome_trace, v4_rome_sites	data/comparison/trace_summary.json (P15)	make_report_v4_figures
v4_rome_sweep	benchmark_results/rome_vs_scalpel/layer_sweep_dev.csv (P15)	make_report_v4_figures
v4_rome_vs_ours	The same CSV, and packs/cross_model_transfer/d_analysis_linear.json (P15, P16)	make_report_v4_figures
r_swap, r_export_doses, r_export_side	b1_swap_primary.json and d_analysis_linear.json in packs/cross_model_transfer/ (P16)	make_report_v2_figures
r_ladder, r_training_gap	d5, d6, d7, d10 and d13 result files in the same pack (P16, P17)	make_report_v2_figures
r_push, r_sae, r_import	s2, s3, s1 and e5 result files in the same pack (P17)	make_report_v2_figures